

P R E F A Ț Ă

Algoritmul este un concept folosit pentru a desemna o mulțime finită de operații, complet ordonată în timp, care pornind de la date de intrare produce într-un timp finit date de ieșire. Cu alte cuvinte, algoritmul redă metoda de rezolvare a unei probleme într-un număr finit de pași.

Programul este reprezentarea unui algoritm într-un limbaj de programare. Sunt cunoscute mai multe limbaje de programare, dezvoltate odată cu evoluția calculatoarelor. O anumită problemă poate fi mai ușor sau mai dificil de codificat într-un anumit limbaj de programare, întrucât multe din limbajele de programare de nivel înalt sunt orientate pe probleme.

Programarea este activitatea de elaborare a unui produs program. Ea are două ramuri importante:

- a) descrierea algoritmilor;
- b) codificarea algoritmilor într-un anumit limbaj de programare.

Descrierea unui algoritm pentru rezolvarea unei probleme se poate face prin scheme logice sau într-un limbaj de descriere a algoritmilor, numit și pseudocod.

La disciplina de "Programarea calculatoarelor" s-a adoptat descrierea algoritmilor în pseudocod, iar ca limbaj de programare se utilizează C/C++. În întreaga activitate legată de această disciplină, se insistă asupra respectării următoarelor etape de rezolvare a unei probleme:

- a) Analiza problemei, care constă în enunțul clar, precis al problemei de rezolvat, specificarea datelor de intrare și ieșire.
- b) Proiectarea programului, care constă în stabilirea metodei de rezolvare și întocmirea proiectului logic.
- c) Implementarea programului, care constă în codificarea într-un limbaj de programare, editarea fișierului sursă, compilarea, editarea de legături, execuția și testarea.
- d) Întocmirea documentației. În cadrul seminarului, fiecare student primește o temă din domeniile generării de submulțimi, calculului numeric, algoritmilor de sortare etc. Documentația va conține următoarele piese:

- enunțul problemei;
- lista variabilelor de intrare și ieșire și modul de reprezentare a lor pe suportul extern;
- prezentarea în limbaj natural a metodei de rezolvare;
- proiectul logic;
- listingul programului sursă;
- rezultatele testării experimentale;
- instrucțiuni de operare.

e) Întreținerea programului.

Respectarea acestor etape conduce la obținerea unor performanțe legate de productivitatea programării și de calitatea produsului program.

De asemenea, atât la curs, cât și la seminar și laborator, se urmărește însușirea unui stil de programare, caracterizat prin atribute care conferă unui program o anumită personalitate:

- evidențierea structurii programului;
- modularizarea programului;
- abstractizarea datelor;
- claritatea programului;
- posibilitatea de modificare ulterioară cu efort mic;
- tratarea erorilor;
- generalitatea soluției.

În acest sens, la codificare, se recomandă următoarele:

- alegerea unor nume simbolice care să reflecte semnificația obiectelor pe care le reprezintă;
- folosirea indentării pentru scoaterea în relief a structurilor de control;
- documentarea programului;
- scrierea unor funcții de înaltă generalitate pentru utilizări ulterioare;
- protecția la erori datorită depășirii dimensiunii tablourilor, a domeniului de valori pentru tipurile de date numerice etc.;
- folosirea variabilelor globale să se facă cât mai puțin posibil.

Prezentul îndrumător conține 11 lucrări de laborator, în care sunt tratate fundamentele limbajului C/C++. Fiecare lucrare conține prezentarea unor noțiuni teoretice cu exemple rulate în BORLAND C/C++ versiunea 4.5. și propuneri de probleme adecvate noțiunilor tratate în lucrare.

Îndrumătorul se adresează studenților din anul I, secția “Știința sistemelor și calculatoarelor” pentru pregătirea și efectuarea lucrărilor de

laborator la disciplina de “Programarea calculatoarelor”, dar este util și celor interesați de perfecționarea în programare și obținerea unui stil de a scrie programe în mod profesionist.

Disciplina de “Programarea calculatoarelor” stă la baza disciplinei “Structuri de date și algoritmi”, unde se tratează listele, arborii, grafurile, tabelele de dispersie și metodele generale de elaborare a algoritmilor.

Menționăm faptul că însușirea gândirii algoritmice și obținerea îndemânării de a scrie programe performante necesită multă muncă individuală. De aceea, se pretinde pregătirea teoretică a lucrărilor de laborator și rezolvarea de către fiecare student acasă a problemelor propuse, iar la orele de laborator numai să se implementeze câteva programe semnificative.

Autorul aduce mulțumiri d-lui Mihai Vintiloiu, d-lui ing. Crișan Valer-Alin și d-lui ing. Coroian-Vlad Răzvan pentru ajutorul dat la editarea lucrărilor de laborator. De asemenea, autorul mulțumește editurii U.T. PRESS pentru publicarea lucrării într-o ținută grafică corespunzătoare.

Cluj-Napoca, 20 iulie 2001

Prof. dr. ing. IGNAT IOSIF

C U P R I N S

1. FUNCȚII DE INTRARE/IEȘIRE STANDARD	9
2. EXPRESII	17
3. INSTRUCȚIUNI	22
4. FUNCȚII	35
5. PROGRAMAREA MODULARĂ	45
6. POINTERI	54
7. RECURSIVITATE	67
8. ȘIRURI DE CARACTERE	74
9. TIPURILE DE DATE STRUCTURĂ, UNIUNE ȘI ENUMERARE	82
10. PRELUCRAREA FIȘIERELOR DE CĂTRE NIVELUL INFERIOR AL S.G.F.	93
11. PRELUCRAREA FIȘIERELOR DE CĂTRE NIVELUL SUPERIOR AL S.G.F.	105
BIBLIOGRAFIE	115

FUNCȚII DE INTRARE/IEȘIRE STANDARD

1. Conținutul lucrării

În lucrare sunt prezentate funcțiile I/E standard, adică funcțiile din biblioteca compilatorului C/C++, care realizează citirea/scrierea din/în fișierele standard de I/E.

1. Considerații teoretice

Terminalul standard este terminalul de la care s-a lansat programul. Terminalului standard îi sunt atașate două fișiere: de intrare (stdin) și de ieșire (stdout). Ele sunt fișiere secvențiale.

Funcțiile din biblioteca compilatorului C/C++ utilizate mai frecvent pentru operațiile de I/E sunt:

- pentru intrare: **getch**, **getche**, **gets**, **scanf**, **sscanf** ;
- pentru ieșire: **putch**, **puts**, **printf**, **sprintf**.

la care se mai adaugă macrourele **getchar** pentru intrare și **putchar** pentru ieșire.

2.1. Funcțiile getch, getche și putch

Funcția **getch** citește fără ecou un caracter prin apăsarea unei taste. Tasta poate avea un corespondent ASCII sau o funcție specială. În primul caz funcția returnează codul ASCII al caracterului. În al doilea caz, funcția se apelează de două ori: prima dată returnează valoarea zero, iar a doua oară returnează o valoare specifică tastei acționate.

Funcția **getche** este analogă cu funcția **getch**, realizând însă citirea cu ecou.

Apelul funcțiilor **getch** și **getche** conduce la așteptarea apăsării unei taste.

Funcția **putch** afișează pe ecranul terminalului un caracter corespunzător codului ASCII transmis ca parametru. Caracterele imprimabile au codul ASCII în intervalul [32,126]. Pentru coduri în afara acestui interval se afișează diferite imagini. Funcția returnează valoarea parametrului de la apel.

Prototipurile acestor trei funcții se găsesc în fișierul **conio.h** și sunt:

```
int getch(void);
int getche(void);
int putch(int ch);
```

Exemplu de utilizare:

```
/* Programul L1Ex1.cpp */

#include <conio.h>
main()
{
    putch(getch());
    getch();
}
```

2.2. Funcțiile gets și puts

Funcția **gets** citește cu ecou de la terminalul standard un șir de caractere ale codului ASCII, la adresa specificată drept parametru al funcției. Din funcție se revine la:

- citirea caracterului '\n' (newline), caracter care este transformat în caracterul '\0' (null). În acest caz funcția returnează adresa de început a zonei de memorie în care se păstrează caracterele;
- citirea sfârșitului de fișier (CTRL/Z), funcția returnând valoarea zero.

Funcția **puts** afișează la terminalul standard un șir de caractere corespunzând codului ASCII de la adresa transmisă ca parametru.

Caracterul ‘\0’ este interpretat ca ‘\n’. Funcția returnează codul ultimului caracter afișat sau –1 în caz de eroare.

Prototipurile funcțiilor se găsesc în fișierul **stdio.h** și sunt:

```
char *gets (char *s);  
int puts (const char *s);
```

Exemplu de utilizare:

```
/* Programul L1Ex2.cpp */  
  
#include <stdio.h>  
#include <conio.h>  
main  
{  
    char s{200};  
    printf("\nIntroduceți un șir de caractere urmat de  
        ENTER\n");  
    gets(s);  
    printf("\nSirul de caractere introdus\n");  
    puts(s);  
    getch();  
}
```

2.3. Funcțiile **scanf** și **printf**

Funcția **scanf** are rolul de a introduce date tastate de la terminalul standard sub controlul unor formate. Datele introduse sunt convertite din formatele lor externe în formate interne și sunt păstrate la adresele specificate la apel. Datele introduse se termină cu apăsarea tastei ENTER.

Prototipul funcției **scanf** se găsește în fișierul **stdio.h** și este:

```
int scanf(const char *format [,adresa,..]);
```

Ea returnează numărul de câmpuri de la intrare introduse corect sau valoarea EOF(-1) în cazul întâlnirii sfârșitului de fișier (CTRL/Z).

Formatul este specificat ca un șir de caractere. El conține specificatorii de format, care definesc conversiile din formate externe în formate interne. Un specificator de format este alcătuit din:

- caracterul %;
- opțional caracterul *, care indică faptul că data prezentă la intrare nu se atribuie nici unei variabile;
- opțional un număr zecimal, care definește lungimea maximă a câmpului controlat de format;
- 1 sau 2 litere, care definesc tipul conversiei.

Câmpul controlat de format începe cu primul caracter curent care nu este alb și se termină, după caz:

- a) la caracterul după care urmează un caracter alb;
- b) la caracterul care nu corespunde tipului de conversie;
- c) la caracterul la care se ajunge la lungimea maximă a câmpului.

Datele se citesc efectiv după apăsarea tastei ENTER. Adresa unei variabile se specifică prin **&nume_variabilă**.

Literele care definesc tipul conversiei sunt:

Litera	Tipul datei citite
<i>c</i>	char
<i>s</i>	șir de caractere
<i>d</i>	întreg zecimal
<i>o</i>	întreg octal
<i>x, X</i>	întreg hexazecimal
<i>u</i>	unsigned
<i>f</i>	float
<i>ld, lo, lx, lX</i>	long
<i>lu</i>	unsigned long
<i>lf/ Lf</i>	double/long double

Funcția **printf** este folosită pentru afișarea unor date pe ecranul terminalului standard sub controlul unor formate. Datele sunt convertite din format intern în formatul extern specificat.

Prototipul funcției **printf** se găsește în fișierul **stdio.h** și este:

int printf(const char *format [,expresie, ...]);

Formatul este dat ca un șir de caractere. El are în structura sa succesiuni de caractere (care se afișează) și specificatori de format.

Un specificator de format conține:

- caracterul %;
- opțional caracterul minus -, care specifică cadrarea datei în stânga câmpului (implicit cadrarea se face în dreapta);
- opțional un număr zecimal, care definește dimensiunea minimă a câmpului în care se afișează data;
- opțional un punct urmat de un număr zecimal, care specifică precizia de afișare a datei;
- una sau două litere, care definesc tipul conversiei. Față de literele prezentate la scanf apar literele e și E pentru afișarea datelor float sau double sub formă de exponent, g și G pentru afișarea datelor sub forma de exponent sau nu, astfel ca data afișată să ocupe un număr minim de caractere.

Funcția returnează numărul de caractere (octeți) afișate la terminal sau -1 în caz de eroare.

Exemple de folosire:

```
/* Programul L1Ex3.cpp */

#include <stdio.h>
#include <conio.h>
main()
{
    int a;
    float b,c;
    printf("\nIntroduceți o valoare întreagă a=");
    scanf("%5d",&a);
    printf("\nIntroduceți o valoare reală b=");
    scanf("%5f",&b);
    c=a+b;
```

```

        printf("\nValoarea c=a+b este: %6.3f\n",c);
        getch();
    }

```

2.4. Funcțiile sscanf și sprintf

Față de funcțiile **scanf** și **printf**, funcțiile **sscanf** și **sprintf** au în plus ca prim parametru adresa unei zone de memorie care conține caractere ASCII. Funcția **sscanf** citește caracterele din această zonă de memorie în loc de zona tampon corespunzătoare fișierului standard de intrare (tastaturii). Funcția **sprintf** depune caracterele în această zonă de memorie în loc de a fi afișate pe ecran.

Prototipurile acestor funcții se găsesc în fișierul **stdio.h** și sunt:

```

int scanf (const char *buffer, const char *format [,adresa, ..]);
int sprintf (char *buffer, const char *format [,adresa, ...]);

```

Exemplu de folosire:

```

/* Programul L1Ex4.cpp */

#include <stdio.h>
#include <conio.h>
main ( )
{
    char s[100], q[100];
    int a,b;
    float c,d;
    printf ("\nIntroduceti în același rând valoarea\n\
        lui a și b despățite între ele prin blanc\n\
        urmate de ENTER\n");
    gets(s);
    sscanf(s,"%d %f", &a, &c);
    printf("\n a=%4d c=%8.3f\n",a,c);
    sprintf(q,"%4d %8.3f\n",a,c);
    sscanf(q,"%d %f",&b,&d);
    printf("\n b=%5d d=%9.4f\n",b,d);
    getch();
}

```

2.5. Macrourele **getchar** și **putchar**

Macroul **getchar** permite citirea cu ecou a caracterelor codului ASCII, deci nu a celor corespunzătoare tastelor speciale. Caracterele tastate se introduc într-o zonă tampon până la acționarea tastei ENTER. La revenire, se returnează codul ASCII al primului caracter introdus, iar la un nou apel, al următorului caracter introdus ș.a.m.d. La întâlnirea sfârșitului de fișier (CTRL/Z) se returnează valoare EOF(-1).

Macroul **putchar** afișează caracterul al cărui cod ASCII s-a transmis.

Macrourele **getchar** și **putchar** sunt definite în fișierul **stdio.h** și au formatele:

```
int getchar(void);  
int putchar (int c);
```

și se apelează exact ca funcțiile **getch** și **putch**.

Exemplu de utilizare:

```
/* Programul L1Ex5.cpp */  
  
#include <stdio.h>  
#include <conio.h>  
main()  
{  
    putchar(getchar());  
    putchar('\n');  
    getch();  
}
```

3. Mersul lucrării

3.1. Se vor executa programele date ca exemplu în lucrare și se vor analiza rezultatele obținute.

3.2. Scrieți un program pentru a verifica modul de execuție a funcției

getch când se apasă o tastă care corespunde unei funcții speciale.

3.3. Scrieți un program pentru a verifica ce se afișează de către funcția putchar atunci când parametrul său este o valoare în afara intervalului [32,126].

3.4. Scrieți un program care afișează codurile ASCII ale caracterelor corespunzătoare tastaturii.

3.5. Scrieți un program care afișează caracterele corespunzătoare codurilor ASCII din intervalul [32,126].

3.6. Scrieți un program care să conțină apelul gets(s), unde s a fost definit ca un tablou.

Verificați ce conține fiecare element al tabloului. De ce caracterul '\n' a fost înlocuit cu '\0'?

3.7. Scrieți un program care citește un șir de litere mici și le afișează sub formă de litere mari.

3.8. Scrieți un program care citește un șir de litere mari și le afișează sub formă de litere mici.

3.9. Scrieți un program care realizează suma, diferența, produsul și împărțirea a două numere reale. Afișarea se va face sub formă tabelară:

x	y	x + y	x - y	x * y	x / y

3.10. Scrieți un program pentru a verifica modul de afișare a valorii lui $\pi = 3.14159265$ cu diferiți descriptorii de format.

3.11. Scrieți un program pentru afișarea unui întreg citit de la tastatură în octal și hexazecimal.

EXPRESII

1. Conținutul lucrării

Scopul lucrării este prezentarea noțiunii de expresie în limbajul C/C++, a operatorilor și a modului de evaluare a unei expresii.

2. Considerații teoretice

2.1 Definirea expresiei

O expresie este formată dintr-un operand sau mai mulți operanzi legați prin operatori.

Un operand poate fi:

- o constantă;
- o constantă simbolică;
- numele unei variabile simple;
- numele unui tablou;
- numele unei structuri;
- numele unui tip;
- o variabilă indexată;
- numele unei funcții;
- referința la elementul unei structuri;
- apelul unei funcții;
- expresie inclusă în paranteze rotunde.

Unui operand îi corespunde o valoare și un tip.

Operatorii pot fi unari sau binari.

La evaluarea unei expresii trebuie să se țină seama de:

- prioritățile operatorilor;
- asociativitatea operatorilor de aceeași prioritate;

- regula conversiilor implicite.

2.2 Operatori

Operatorii limbajului C/C++ sunt împărțiți în următoarele clase:

- operatori aritmetici:
 - operatori unari: +, -
 - operatori binari multiplicativi: *, /, %
 - operatori binari aditivi: +, -
- operatori relaționali: <, <=, >, >=
- operatori de egalitate: ==, !=
- operatori logici: !, &&, ||
- operatori logici pe biți: ~, <<, >>, &, ^, |
- operatori de atribuire: =, /=, *=, %=, +=, -=, <<=, >>=, &=, ^=, |=
- operatori de incrementare: ++, --
- operatori de forțare tip: (tip) operand
- operatori dimensiune: sizeof
- operatori adresă: &
- operatori paranteză: (), []
- operatori condiționali: ?, :
- operatorul virgulă: ,
- operatorul de dereferențiere: *
- operatorul de acces la componenta unei structuri: ., ->

Observație: În limbajul C++ s-au mai introdus câțiva operatori:

- operatorul de rezoluție: ::
- operatorul pentru tipul referință: &
- operatorul de alocare/dealocare dimensiune a memoriei heap: new/delete

Prioritățile operatorilor, în ordine descrescătoare, sunt prezentate în tabelul de mai jos:

Prioritățile	Operatori
1	() [] • ->
2	+(unar) -(unar) &(unar) *(unar) ++ -- (tip) sizeof ! ~
3	*(binar) / %
4	+(binar) -(binar)
5	<< >>
6	< <= > >=
7	= = !=
8	&(binar)
9	^
10	
11	&&
12	
13	? :
14	= <<= >>= += -= *= /= %= &= ^= =
15	,

Operatorii din aceeași linie au aceeași prioritate.

În ceea ce privește asociativitatea operatorilor, se precizează că ei se asociază de la stânga la dreapta cu excepția operatorilor unari, condiționali și de atribuire, care se asociază de la dreapta la stânga.

2.3. Regula conversiilor implicite

Regula conversiilor implicite constă în următoarele:

- dacă un operator binar se aplică la operanzi de același tip, atunci rezultatul va avea tipul comun al operanzilor;
- dacă un operator binar se aplică la doi operanzi de tipuri diferite, atunci operandul de tip inferior se convertește implicit spre tipul superior al celuilalt operand, iar rezultatul va avea tipul superior.

Ordinea descrescătoare a priorității tipurilor este următoarea:

- long double;

- double;
- float;
- unsigned long;
- long;
- int.

3. Mersul lucrării

3.1. Să se scrie un program pentru calculul valorii $z = x^{**}y$, x și y fiind de tipul double.

3.2. Explicați deosebirea între împărțirea reală și cea întreagă.

3.3. Să se scrie un program care citește un unghi în grade sexagesimale și calculează valoarea funcției sinus, cosinus și tangentă.

3.4. Să se scrie un program care citește un număr natural în intervalul [1600, 4900] ce reprezintă un an și verifică dacă este bisect sau nu.

3.5. Folosind expresii condiționale, să se scrie un program care citește valoarea reală a lui x și calculează valoarea funcției:

$$f(x) = \begin{cases} x^2 + 4x + 4 & \text{daca } x < -2 \\ 0 & \text{daca } x = 0 \\ x^2 + 5x & \text{daca } x > -2 \end{cases}$$

3.6. Să se scrie un program care citește un număr real x , ce reprezintă măsura unui unghi în radiani, și îl transformă în grade, minute și secunde sexagesimale.

3.7. Să se scrie un program care simulează funcționarea unui numărător de tip ceas (se indică ora, minutul și secunda).

3.8. Scrieți un program pentru a indica numărul de octeți ocupați în memorie de tipurile de date din limbajul C/C++.

3.9. Converteți în binar, prin calcul, anul nașterii și anul curent.

Arătați cum se reprezintă ca o dată de tip int. Efectuați operațiile de deplasare stânga cu 4 biți, dreapta cu 2 biți, complement față de 1 asupra lor, precum și operațiile pe biți &, ^, |, având ca operanzi cele două date. Scrieți un program pentru a verifica corectitudinea calculelor dumneavoastră.

3.10. Scrieți un program ce efectuează operații aritmetice asupra a două date de tip întreg și real. Executați-l pentru valori care conduc la rezultat în afara limitelor de reprezentare internă. Ce se întâmplă în acest caz?

INSTRUCȚIUNI

1. Conținutul lucrării

În lucrare sunt prezentate principalele instrucțiuni simple și structurate din limbajul C/C++: instrucțiunea expresie, instrucțiunea vidă, instrucțiunea compusă, instrucțiunea **if**, instrucțiunea **switch** și instrucțiunile repetitive.

2. Considerații teoretice

Programul structurat este un program care are o structură de control realizată numai cu:

- structura secvențială;
- structura alternativă și selectivă;
- structura repetitivă.

În limbajul C/C++ mai există instrucțiunile **return**, **break**, **continue** și **goto**, care asigură o flexibilitate mare în scrierea de programe.

2.1. Instrucțiunea expresie

Instrucțiunea expresie are formatul:

expresie;

adică după expresie se scrie “;”.

Ea se utilizează ca instrucțiune de atribuire sau ca instrucțiune de apel a unei funcții.

Exemplu de utilizare:

```

/* Programul L3Ex1.cpp */

/* Programul afiseaza maximul dintre 2 intregi */
#include <conio.h>
#include <stdio.h>
main()
{
    int a,b,c;
    printf("\nIntroduceti doi intregi a si b\n");
    scanf("%d %d",&a,&b);
    c=a>b?a:b;
    printf("\nMaximul dintre a=%d si b=%d este c=%d\n",a,b,c);
    getch();
}

```

2.2.Instrucțiunea vidă

Instrucțiunea vidă se reduce la punct și virgulă, fără a avea vreun efect. Ea se utilizează acolo unde se cere prezența unei instrucțiuni, dar de fapt nu trebuie să se execute ceva (de exemplu în instrucțiunile repetitive).

Exemplu de utilizare:

```
for(i = 0, s = 0; i < n; s = s + a[i], ++i);
```

2.3.Instrucțiunea compusă

Instrucțiunea compusă este o succesiune de instrucțiuni incluse între acolade, eventual precedate de declarații (valabile numai în acest loc):

```

{
    declarații;
    instrucțiuni;
}

```

Instrucțiunea compusă se utilizează acolo unde este nevoie conform sintaxei de o singură instrucțiune, dar procesul de calcul necesită mai multe instrucțiuni.

Exemplu de utilizare este dat în programul L3Ex2.cpp :

```

/* Programul L3Ex2.cpp */

/* Calculul radacinilor ecuatiei  $a \cdot x^2 + b \cdot x + c = 0$  */
#include <stdio.h>
#include <conio.h>
#include <math.h>
main()
{
    float a,b,c,delta,x1,x2;
    printf("\nIntroduceti a,b,c\n");
    scanf("%f %f %f",&a,&b,&c);
    if (a!=0)
    {
        delta=b*b-4*a*c;
        if (delta >= 0) {
            x1=(-b-sqrt(delta))/(2*a);
            x2=(-b+sqrt(delta))/(2*a);
            printf("\nEcuatia are radacinile x1=%g si
                    x2=%g\n",x1,x2);
        }
        else {
            x1=-b/(2*a);
            x2=sqrt(-delta)/(2*a);
            printf("\nEcuatia are radacinile complex conjugate:\n
                    x1=%g - j*%g si x2= %g+ j*%g\n",x1,x2,x1,x2);
        }
    }
    else printf("\nEcuatia nu este de ordinul 2 (a=0)\n");
    getch();
}

```

2.4 Instrucțiunea if

Instrucțiunea if are două formate:

a) **if (expresie)**

instrucțiune

b) if (expresie)
 instrucțiune_1
 else instrucțiune_2

Efectul ei este următorul:

Se evaluează expresia “expresie”.

Dacă rezultatul expresiei este **true** se execută în cazul a) instrucțiunea “instrucțiune” și în cazul b) “instrucțiune_1” și apoi se trece la instrucțiunea imediat următoare instrucțiunii **if**.

Dacă rezultatul expresiei este **false** se trece în cazul a) la instrucțiunea imediat următoare instrucțiunii **if**, iar în cazul b) se trece la execuția “instrucțiune_2” și apoi se trece la instrucțiunea imediat următoare instrucțiunii structurate **if**.

Observații:

a) instrucțiunile “instrucțiune”, “instrucțiune_1”,
 “instrucțiune_2” pot conține instrucțiuni de salt la alte instrucțiuni decât cea următoare instrucțiunii **if**.

b) instrucțiunea **if** poate conține alte instrucțiuni **if**. Trebuie atenție la îmbinarea lui **else**, în sensul de a ști la care **if** aparține.

Exemplu de utilizare: Programul L3Ex2.cpp (a se vedea punctul 2.3).

2.5 Instrucțiunea switch

Instrucțiunea **switch** are următoarea sintaxă:

```
switch ( expresie )  
{  
  case C1: sir_instrucțiuni_1;  
    break;  
  case C2: sir_instrucțiuni_2;  
    break;  
  .....  
  case Cn: sir_instrucțiuni_n;  
    break;  
  default: sir_instrucțiuni  
}
```

Efectul instrucțiunii **switch** este următorul:

- a) se evaluează “expresie”;
- b) se compară pe rând rezultatul evaluării cu constantele C1, C2, ..., Cn. Dacă rezultatul evaluării coincide cu constanta Ci se execută instrucțiunile “sir_instrucțiuni_i” și apoi se trece la instrucțiunea imediat următoare **switch**-ului. Dacă rezultatul evaluării nu coincide cu nici una din constantele C1, C2, ..., Cn se execută instrucțiunile “sir_instrucțiuni” aflate după “**default**”.

Observații:

- a) Alternativa **default** este opțională. Dacă nu este prezentă, în cazul în care rezultatul expresiei “expresie” nu coincide cu nici o constantă Ci, instrucțiunea **switch** nu are nici un efect.
- b) Dacă **break** nu este prezentă, atunci se execută și șirurile de instrucțiuni imediat următoare, până la întâlnirea unei instrucțiuni **break** sau până la terminarea instrucțiunii **switch**.
- c) Instrucțiunea structurată **switch** poate fi înlocuită prin instrucțiuni **if** imbricate.

Exemplu de utilizare:

```
/* Programul L3Ex3.cpp */

/* Operatii cu numere intregi de forma
                                OPERAND1operatorOPERAND2 */
#include <stdio.h>
#include <conio.h>
#include <stdlib.h>
#define INFINIT 0x7fff
main()
{
    int operand1,operand2,rezultat;
    char operatie;
    printf("\nScrieti expresia fara spatii intre operanzi și\
        operator\n");
    scanf("%d%c%d",&operand1,&operatie,&operand2);
    switch(operatie)
    {
        case '+': rezultat=operand1+operand2;
```



```

        break;

    case '-': rezultat=operand1-operand2;
        break;
    case '*': rezultat=operand1*operand2;
        break;
    case '/': if (operand2!=0) rezultat =
                operand1/operand2;
        else if (operand1 >0) rezultat=INFINT;
        else rezultat=-INFINT;
        break;
    default: exit(1);
};
printf("\n%d %c %d = %d\n", operand1, operatie,
        operand2, rezultat);
getch();
}

```

2.6 Instrucțiunea while

Formatul instrucțiunii **while** este următorul:

```

while ( expresie )
    instrucțiune

```

Efectul instrucțiunii **while** este următorul:

- se evaluează “expresie”;
- dacă rezultatul este **true** se execută corpul său (“instrucțiune”) și se revine la pasul a). Dacă rezultatul este **false** se trece la execuția instrucțiunii imediat următoare instrucțiunii **while**.

Observații:

- În cazul în care expresie este **false** de la început, atunci “instrucțiune” nu se execută niciodată.
- În cadrul corpului instrucțiunii **while** este nevoie de existența unor instrucțiuni de modificare a variabilelor care intră în “expresie”.

Exemplu de utilizare:

```

/* Programul L3Ex4.cpp */

/* Calculul c.m.m.d.c. si a c.m.m.m.c
   a doua numere naturale a si b */
#include <conio.h>
#include <stdio.h>
main()
{
    int a,b,a1,b1,cmmdc,cmmmc,rest;
    printf("Introduceti a=");
    scanf("%d",&a);
    printf("Introduceti b=");
    scanf("%d",&b);
    /* Aflarea c.m.m.d.c. */
    a1=a;b1=b;
    while ((rest=a1%b1)!=0)
    {
        a1=b1;
        b1=rest;
    };
    cmmdc=b1;
    cmmmc=a*b/cmmdc;
    clrscr();
    printf("a=%d b=%d cmmdc(a,b)=%d cmmmc=%d", a, b,
           cmmdc, cmmmc);
    getch();
}

```

2.7 Instrucțiunea for

Formatul instrucțiunii **for** este următorul:

```

for ( expr1; expr2; expr3 )
    instrucțiune

```

unde:

- expr1, expr2, expr3 sunt expresii;
- instrucțiune este corpul instrucțiunii.

Descrierea efectului instrucțiunii **for**, cu ajutorul instrucțiunii **while** este următorul:

```
expr1;
while ( expr2 ) {
    instrucțiune;
    expr3;
}
```

Observație: expr1, expr2, expr3 pot fi vide, însă caracterele “;” nu pot lipsi.

Exemplu de utilizare:

```
/* Programul L3Ex5.cpp */

/* Calculul mediei aritmetice a n numere reale */
#include <conio.h>
#include <stdio.h>
main()
{
    float a[100],media,suma;
    int i,n;
    printf("\nIntroduceti nr.de elemente n=");
    scanf("%d",&n);
    printf("\nIntroduceti elementele sirului\n");
    for(i=0,suma=0;i<n;++i)
    {
        printf("a[%2d]=",i);
        scanf( "%f",&a[i]);
        suma+=a[i];
    };
    media=suma/n;
    printf("\nMEDIA=%g\n",media);
    getch();
}
```

2.8 Instrucțiunea do-while

Instrucțiunea **do-while** este instrucțiunea ciclică cu test final.
Formatul ei este următorul:

```
do  
    instrucțiune  
while ( expresie );
```

Efectul ei este descris cu instrucțiunea **while** astfel:

```
instrucțiune;  
while( expresie )  
    instrucțiune;
```

Se observă că corpul ciclului se execută cel puțin o dată.

Exemplu de utilizare:

```
/* Programul L3Ex6.cpp */  
  
/* Calculul c.m.m.d.c. si a c.m.m.m.c  
   a doua numere naturale a si b */  
#include <conio.h>  
#include <stdio.h>  
main()  
{  
    int a,b,a1,b1,cmmdc,cmmmc,rest;  
    printf("Introduceti a=");  
    scanf("%d",&a);  
    printf("Introduceti b=");  
    scanf("%d",&b);  
    /* Aflarea c.m.m.d.c. */  
    a1=a;b1=b;  
    do{  
        rest=a1%b1;  
        a1=b1;  
        b1=rest;  
    }  
    while (rest!=0);  
    cmmdc=a1;
```

```

    cmmmc=a*b/cmmdc;
    clrscr();
    printf("a=%d b=%d cmmdc(a,b)=%d
           cmmmc=%d",a,b,cmmdc,cmmmc);
    getch();
}

```

2.9 Instrucțiunile continue și break

Instrucțiunile **continue** și **break** se pot utiliza numai în corpul unui ciclu.

Instrucțiunea **continue** abandonează iterația curentă și se trece la execuția pasului de reinițializare în cazul instrucțiunii **for**, respectiv la revalidarea expresiei care stabilește continuarea sau terminarea ciclului în cazul instrucțiunilor **while** și **do-while**.

Instrucțiunea **break** termină ciclul și se trece la instrucțiunea imediat următoare celei repetitive (**for**, **while**, **do-while**).

2.10. Instrucțiunea goto

Instrucțiunea **goto** este utilizată pentru saltul dintr-un punct al unei funcții (chiar și dintr-un ciclu) în alt punct al aceleiași funcții, respectiv la o instrucțiune etichetată.

Eticheta este un nume urmat de caracterul “:”

nume:

Formatul instrucțiunii **goto** este:

goto eticheta;

Exemplu:

```

...
goto alfa;
...
alfa: if ( ) ...
...

```

2.11. Funcția standard exit

Prototipul funcției standard **exit** este descris în fișierele **stdlib.h** și **process.h** și este:

void exit(int cod);

Funcția **exit** are ca scop terminarea forțată a programului. Codul de ieșire folosit este zero pentru terminare normală și alte valori pentru terminare datorată unor erori.

3. Mersul lucrării

3.1. Se vor analiza și executa programele date ca exemplu în lucrare.

3.2. De la tastatură se citesc 4 perechi de numere reale, care reprezintă în coordonate rectangulare vârfurile unui patrulater. Să se stabilească natura acestui patrulater.

3.3. De pe mediul de intrare sunt citite elementele reale ale unui șir de dimensiunea n . Să se găsească valoarea minimă și valoarea maximă dintre elementele șirului și poziția lor.

3.4. Să se scrie un program pentru generarea tuturor numerelor prime mai mici sau egale cu un număr natural n .

3.5. Se citește un număr natural n . Să se găsească cel mai mare pătrat perfect mai mic sau egal cu n . Aceeași problemă, dar să se indice numărul prim cel mai mic, dar mai mare sau egal cu numărul citit.

3.6. De pe mediul de intrare se citește un număr natural n . Să se verifice dacă numărul respectiv este palindrom.

3.7. De pe mediul de intrare se citesc cifrele hexazecimale ale unui număr întreg în baza 16. Să se calculeze și să se afișeze reprezentarea numărului în baza 10.

3.8. Se citește gradul și coeficienții polinomului $p(x)=a_0 + a_1x^1 + \dots + a_nx^n$. Să se calculeze valoarea polinomului în $x = x_0$ (x_0 se citește).

3.9. Să se scrie un program pentru efectuarea operațiilor de adunare, scădere, înmulțire și împărțire între două polinoame:

$$A(x) = a_0 + a_1x^1 + \dots + a_nx^n$$

$$B(x) = b_0 + b_1x^1 + \dots + b_mx^m$$

Gradele și coeficienții reali ai polinoamelor se citesc de pe mediul de intrare.

3.11. Se dă un sistem de n ecuații liniare cu n necunoscute. Să se scrie un program de rezolvare a sistemului, folosind o metoda numerică.

3.12. Sa se calculeze polinoamele $P(x)$ și $Q(x)$ din relația:

$$\frac{Q(X)}{P(X)} = \sum_{i=1}^n \frac{a_i}{b_i x + c_i}$$

Valorile n , a_i , b_i , c_i se citesc de la tastatură.

3.13. Se dă un șir de n elemente reale ordonate crescător. Să se verifice dacă o valoare citită x se găsește în șir și să se indice poziția sa.

3.14. Se dă un șir de n numere întregi. Să se extragă subșirul de dimensiune maximă, ordonat crescător.

3.15. Pentru elaborarea unui test de aptitudini se dispune de un set de n întrebări, fiecare întrebare i fiind cotate cu un număr de p_i puncte. Să se elaboreze toate chestionarele având q întrebări, fiecare chestionar totalizând între a și b puncte. Întrebările sunt date prin număr și punctaj.

3.16. Se dau 2 șiruri de n și respectiv m elemente de tip întreg. Să se calculeze:

- a) șirul ce conține elementele comune ale celor două șiruri;
- b) șirul ce conține toate elementele celor două șiruri luate o singura dată;
- c) șirul ce conține elementele primului șir din care au fost eliminate elementele comune.

3.17. Se dă un număr real a în baza 10. Să se scrie programul de conversie a lui în baza B , $B \leq 16$.

3.18. Se dă un număr natural n .

a) Să se găsească numărul obținut prin eliminarea cifrelor care apar de mai multe ori în număr.

b) Să se găsească numărul obținut prin interschimbarea între ele a primei cifre cu ultima, a celei de a doua cu penultima ș.a.m.d.

c) Să se găsească cel mai mare număr ce se poate obține din cifrele sale.

3.19. Se dă o matrice de $n \times n$ elemente 0 și 1. Să se stabilească dacă matricea respectivă este simetrică.

3.20. De pe mediul de intrare se citește o propoziție. Să se indice numărul cuvintelor și cuvântul cel mai lung din propoziție.

FUNCTII

1. Conținutul lucrării

În lucrare se prezintă structura unei funcții, apelul unei funcții prin valoare și prin referință, prototipul unei funcții.

2. Considerații teoretice

Un program conține una sau mai multe funcții, dintre care una este funcția principală având numele **main**. Celelalte au un nume dat de programator.

2.1.Structura unei funcții

O funcție are următoarea structură:

```
tip nume (lista_parametrilor_formali)  
{  
    declarații  
    instrucțiuni  
}
```

Primul rând din definiția funcției se numește **antet**, iar restul se numește **corpul funcției**.

În limbajul C/C++ există două categorii de funcții:

- funcții care returnează în punctul de apel o valoare prin instrucțiunea **return expresie**; valoarea având tipul specificat în antet prin “tip”;
- funcții care nu returnează nici o valoare în punctul de apel, tip fiind înlocuit prin cuvântul cheie “**void**”.

Lista parametrilor formali poate conține:

- zero parametri, caz în care antetul funcției se reduce la:
tip nume () sau **tip nume (void)**
- unul sau mai mulți parametri formali, separați între ei prin virgulă. Un parametru formal se indică prin: **tip nume**.

Exemplu:

```
int rezolv_sistem (int n, double a [10] [10], double b[10], double x [10])
```

Prototipul unei funcții se obține scriind punct și virgulă după o construcție identică cu antetul funcției respective sau obținută prin eliminarea numelui parametrilor formali.

Exemplu:

```
int factorial (int n);  
int factorial (int);
```

Funcțiile standard de bibliotecă au prototipurile în diferite fișiere cu extensia **.h**, cum ar fi **stdio.h**, **conio.h**, **math.h** etc. Funcțiile standard de bibliotecă se găsesc în format obiect (extensia **.obj**), și se adaugă în programe în faza de editare de legături. Prototipurile funcțiilor standard se includ în program înainte de apelul lor prin construcția **#include**.

1.1. Apelul unei funcții

O funcție care nu returnează nici o valoare se apelează astfel:

```
nume (lista_parametrilor_efectivi);
```

Correspondența între parametrii formali și cei efectivii este pozițională.

O funcție care returnează o valoare poate fi apelată

- fie printr-o instrucțiune de apel ca mai sus, caz în care valoarea returnată se pierde;
- fie ca un operand al unei expresii, valoarea returnată folosindu-se la evaluarea expresiei respective.

Tipul parametrilor formali și cei actuali se recomandă să fie același. În caz contrar, în limbajul C tipul parametrului efectiv este convertit automat la tipul parametrului formal. În limbajul C++ se utilizează o verificare mai complexă pentru apelul funcțiilor. De aceea se recomandă utilizarea operatorului de conversie explicită (**tip**), adică expresiile cast.

Exemplu:

f((double)n)

Revenirea dintr-o funcție se face fie după execuția ultimei instrucțiuni din corpul funcției, fie la întâlnirea instrucțiunii **return**.

Instrucțiunea **return** are formatele:

return;

sau

return expresie;

Observație:

- a) Dacă tipul expresiei din instrucțiune este cel din antetul funcției, se face conversia automată spre cel al funcției.
- b) Primul format al funcției **return** se folosește în funcțiile care nu returnează nici o valoare.

Transmiterea parametrilor efectivii (actuali) se poate face:

- prin valoare (call by value);
- prin referință (call by reference).

În cazul apelului prin valoare, unui parametru formal *i* se transferă valoarea parametrului efectiv. În acest caz, funcția apelată nu poate modifica parametrul efectiv din funcția care a făcut apelul, neavând acces la el. Programul L4Ex1.cpp ilustrează acest lucru:

```

/*Programul L4Ex1.cpp */

#include <stdio.h>
#include <conio.h>
/* APEL PRIN VALOARE */
/*Procedura de interschimbare intre a si b */
void interschimbare(int a,int b)
{
    int aux;
    printf("\nIn functie la intrare a=%d b=%d\n",a,b);
    aux=a;a=b;b=aux;
    printf("\nIn functie la iesire a=%d b=%d\n",a,b);
}
void main()
{
    int a,b;
    a=2;b=3;
    printf("\nIn main inaintea apelului functiei interschimbare\
        a=%d b=%d\n",a,b);
    interschimbare(a,b);
    printf("\nIn main la revenirea din functia interschimbare\
        a=%d b=%d\n",a,b);
    getch();
}

```

Se va constata că a și b își păstrează vechile valori.

Pentru ca interschimbarea să se producă, este necesară folosirea pointerilor, ca mai jos:

```

/*Programul L4Ex2.cpp */

#include <stdio.h>
#include <conio.h>
/* APEL PRIN VALOARE FOLOSIND POINTERI*/
/*Procedura de interschimbare intre a si b */
void interschimbare(int *a,int *b)
{
    int aux;
    printf("\nIn functie la intrare a=%d b=%d\n",*a,*b);

```

```

    aux=*a;*a=*b;*b=aux;
    printf("\nIn functie la iesire a=%d b=%d\n",*a,*b);
}
void main()
{
    int a,b;
    a=2;b=3;
    printf("\nIn main inaintea apelului functiei interschimbare\
           a=%d b=%d\n",a,b);
    interschimbare(&a,&b);
    printf("\nIn main la revenirea din functia interschimbare\
           a=%d b=%d\n",a,b);
    getch();
}

```

Se va constata că valorile a și b au fost interschimbate între ele. Acest mod de transmitere a parametrilor este tot prin valoare, adică unui pointer i s-a transmis o adresă.

În cazul apelului prin referință, se transmit adresele parametrilor, nu valoarea lor. Acest mod de transmitere este valabil în C++. În acest caz, parametrii formali sunt definiți ca fiind de tip referință.

Același exemplu în acest caz devine:

```

/*Programul L4Ex3.cpp */

#include <stdio.h>
#include <conio.h>
/* APEL PRIN REFERINTA */
/*Procedura de interschimbare intre a si b */
void interschimbare(int& a,int& b)
{
    int aux;
    printf("\nIn functie la intrare a=%d b=%d\n",a,b);
    aux=a;a=b;b=aux;
    printf("\nIn functie la iesire a=%d b=%d\n",a,b);
}

```

```

void main()
{
    int a,b;
    a=2;b=3;
    printf("\nIn main inaintea apelului functiei interschimbare\
           a=%d b=%d\n",a,b);
    interschimbare(a,b);
    printf("\nIn main la revenirea din functia interschimbare\
           a=%d b=%d\n",a,b);
    getch();
}

```

Se va constata că valorile lui a și b au fost interschimbate între ele.

Observație importantă: în cazul în care un parametru efectiv este numele unui tablou, atunci acesta are ca valoare adresa primului element, deci se transmite adresa ca valoare pentru parametrul formal corespunzător. În acest caz, deși apelul s-a făcut prin valoare, funcția respectivă poate modifica elementele tabloului al cărui nume s-a folosit ca parametru efectiv.

Drept exemplu de folosire a funcțiilor, în continuare se prezintă un program care realizează câteva operații asupra a două polinoame.

/*Programul L4Ex4.cpp */

/* Operatii asupra polinoamelor

Un polinom are forma $P(x)=p[0]+p[1]*x+ p[2]*x^2 +...p[n]* x^n$ */

#include <conio.h>

#include <stdio.h>

define GRADMAX 20

**void produs(int n,float a[], int m,float b[],
 int *p,float c[])**

```

{
    int i,j;
    *p=n+m;
    for(i=0;i<=n+m;i++) c[i]=0.0;
    for(i=0;i<=n;i++)

```

```

    for(j=0;j<=m;j++)
        c[i+j]+=a[i]*b[j];
}
void impartire(int n, float a[],int m,float b[],
    int *grad_cat,float cat[], int *grad_rest, float rest[])
{
    int i,j,k;
    if (n<m) {
        *grad_cat=0;cat[0]=0.0;
        *grad_rest=m;rest=cat;
    }
    else {
        *grad_cat=n-m;*grad_rest=m-1;
        for(i=n-m,j=n;i>=0;i--,j--)
        {
            cat[i]=a[j]/b[m];
            for (k=m;k>=0;k--)
                a[i+k]=a[i+k]-cat[i]*b[k];
            a[j]=0;
        };
        for(i=0;i<=m-1;i++)
            rest[i]=a[i];
    }
}

void citire_polinom(int *n,float a[])
{
    int i;
    printf("\nIntroduceti gradul polinomului ");
    scanf("%d",n);
    for(i=0;i<=*n;i++)
    {
        printf("\na[%d]=",i);
        scanf("%f",&a[i]);
    };
    printf("\n");
}

```

```

float val_polinom(float x,int n,float a[])
{
    int i;
    float v;
    v=0.0;
    for(i=n;i>=0;i--)
        v=v*x+a[i];
    return v;
}

void afis_polinom(int n,float a[],char c)
{
    int i;
    printf("\n%c[x]=%g",c,a[0]);
    for(i=1;i<=n;i++)
        printf("+%g*x^%d",a[i],i);
    printf("\n");
}

void main()
{
    int n,m,grad_r,grad_cat,grad_rest;
    float x, v,p[GRADMAX+1],q[GRADMAX+1],r[GRADMAX+1],
        cat[GRADMAX+1],rest[GRADMAX+1];

    clrscr;
    citire_polinom(&n,p);afis_polinom(n,p,'P');
    citire_polinom(&m,q);afis_polinom(m,q,'Q');
    printf("\nIntroduceti x=");scanf("%f",&x);
    v=val_polinom(x,n,p);
    printf("Val.Polinomului p pentru x=%f este %f",x, v);
    getch();
    produs(n,p,m,q,&grad_r,r);
    printf("\nR[x]=P[x]*Q[x]\n");
    afis_polinom(grad_r,r,'R');
    getch();
    impartire(n,p,m,q,&grad_cat,cat,&grad_rest,rest);
    printf("\nREZULTATUL IMPARTIRII P[x]/Q[x]=>catul C[x] și\
        restul R[x]\n");
    afis_polinom(grad_cat,cat,'C');
    afis_polinom(grad_rest,rest,'R');
}

```



```

getch();
printf("\nATENȚIE! Polinomul p este modificat\n");
afis_polinom(n,p,'P');
getch();
}

```

3. Mersul lucrării

3.1. Se va analiza modul de transmitere a parametrilor efectivi în programele din lucrare date ca exemplu.

În continuare se vor scrie programe pentru rezolvarea următoarelor probleme, folosind funcții și diverse moduri de transmitere a parametrilor. Se va evita folosirea variabilelor globale.

3.2. De pe mediul de intrare se citește gradul unui polinom și coeficienții săi, care sunt numere întregi.

$$P(x) = p_0 + p_1x^1 + \dots + p_nx^n$$

în care p_0 este nenul.

Știind că polinomul admite numai rădăcini întregi simple, să se găsească rădăcinile polinomului.

3.3. De pe mediul de intrare se citește n și perechile de numere întregi (x_i, y_i) , $i = \overline{1, n}$ reprezentând o relație binară R peste mulțimea M .

a) Admițând că fiecare element din mulțimea M apare în cel puțin o pereche dintre cele citite, să se determine mulțimea M .

b) Să se verifice dacă relația R este o relație de echivalență (reflexivă, simetrică și tranzitivă).

3.4. Se dau două șiruri de caractere care reprezintă numere întregi zecimale foarte mari. Să se scrie un program de efectuare a operațiilor aritmetice asupra lor.

3.5. Să se scrie funcțiile pentru adunarea, scăderea și înmulțirea a două matrice și apoi să se realizeze calculul $A = B * C - 2 * (B + C)$, unde B și C sunt două matrice pătratice de ordinul n .

3.6. Să se scrie funcția care realizează operațiile aritmetice asupra a două matrice rare (matricea rară este o matrice de dimensiune mare, care are multe elemente nule).

3.7. Fiind date anul, luna, ziua, să se scrie o funcție care să returneze a câtea zi din an este ziua respectivă și câte zile au mai rămas din anul respectiv.

3.8. Să se scrie o funcție care primind ca parametru un număr roman sub forma unui șir de caractere, returnează numărul respectiv ca număr arab în baza 10.

3.10. Să se scrie o funcție care primind ca parametru un număr arab în baza 10, calculează șirul de caractere ce reprezintă numărul respectiv sub formă romană.

3.11. De pe mediul de intrare se citește un număr întreg, multiplu de 100. Să se găsească numărul minim de bancnote românești necesare pentru plata sumei respective.

PROGRAMAREA MODULARĂ

1. Conținutul lucrării

În lucrare sunt prezentate conceptele de modul, programare modulară și vizibilitate a variabilelor

2. Considerații teoretice

2.1. Noțiunea de modul

Modulul sursă este o parte a textului sursă al programului, care se compilează separat de restul textului sursă a programului.

Modulul obiect este rezultatul compilării unui modul sursă.

Un modul sursă conține funcții înrudite, în sensul ca ele concură la rezolvarea unei subprobleme.

De fapt modulele corespund subproblemelor rezultate în urma proiectării top-down a unei probleme complexe.

Programarea modulară este stilul de programare care are la bază utilizarea de module. Prin acest stil de programare se poate pune în valoare posibilitățile de “ascundere” a datelor și procedurilor împotriva unor accese neautorizate din alte module. Astfel, datele statice declarate în afara funcțiilor modulului, pot fi utilizate în comun de acestea, dar nu pot fi accesate de către funcțiile din alte module.

Recomandarea care se face în scrierea unui program complex este de a-l modulariza, permițând lucrul în echipă.

Modularizarea unui program duce la punerea la punct a programului (implementarea și testarea) mai rapidă.

Modulele puse la punct pot fi utilizate ulterior pentru rezolvarea altor probleme.

În rezolvarea unor probleme complexe, programul executabil poate fi obținut în următoarele moduri:

- a) Se scriu mai multe fișiere sursă, fiecare sursă constituind un modul sursă. Evident, fiecare modul sursă este pus la punct separat. Cu ajutorul construcției

#include “specificator de fișier”

sunt incluse textele sursă fie în modulul care conține funcția principală **main**, fie într-un fișier care conține numai includerile tuturor modulelor, inclusiv a modulului care conține funcția principală **main()**.

În felul acesta se obține de fapt un singur program sursă, care va fi compilat, linkeditat și executat.

- b) Se scriu mai multe module sursă. Se compilează separat obținându-se mai multe module obiect (având extensia .obj). Se linkeditează aceste module obiect, obținându-se fișierul executabil. Modulele sursă pot fi compilate separat și linkeditate folosind un fișier de tip Project. Acesta se editează utilizând meniul Project al mediului Turbo C++.

2.2. Domeniul de valabilitate al variabilelor

2.2.1. Variabile globale

Variabilele globale sunt definite la începutul unui fișier sursă, deci înaintea primei funcții. Ele sunt variabile vizibile din locul respectiv până la sfârșitul fișierului sursă respectiv. Dacă programul are mai multe fișiere sursă, o variabilă globală definită într-un fișier sursă poate fi utilizată în celelalte, dacă este declarată ca externă. Declararea unei variabile externe se poate face:

- după antetul unei funcții, caz în care variabila globală este valabilă numai în acea funcție;
- la începutul fișierului sursă, adică înaintea primei funcții, caz în care este valabilă pentru toate funcțiile din acel fișier.

Observație: Se recomandă ca variabilele externe să fie declarate în fiecare funcție unde se utilizează, evitând erorile care pot apărea prin mutarea ulterioară a unei funcții în alt modul

Variabilele globale sunt alocate la compilare, într-o zonă de memorie specială.

2.2.2. Variabilele locale

Variabilele declarate într-o funcție sau într-o instrucțiune compusă au valabilitate numai în unitatea declarată.

Ele pot fi:

a) **automatic** – care sunt alocate pe stivă la execuție. Ele își pierd existența la revenirea din funcție sau la terminarea instrucțiunii compuse. Declararea lor este cea obișnuită (int a,b,c; double x; etc.);

b) **static** - care sunt alocate la compilare într-o zonă specială. Declararea se face cu ajutorul cuvântului cheie **static** înaintea tipului variabilei. Exemplu:

static int x,y,z;

Declararea unei variabile statice poate fi făcută:

- la începutul fișierului sursă (deci înaintea primei funcții). În acest caz variabila statică respectivă este valabilă în tot fișierul sursă respectiv, dar nu poate fi declarată ca externă în alte fișiere;
- în corpul unei funcții, caz în care este valabilă numai în ea sau în instrucțiunea compusă unde a fost declarată.

c) **variabile registru** – care sunt alocate în registrele procesorului. Ele pot fi numai variabile int, char și pointer. Se recomandă declararea ca variabile registru, variabilele des utilizate în funcția respectivă. Numărul variabilelor registru este limitat. Dacă s-au declarat mai multe, cele care nu pot fi alocate în registre vor fi alocate pe stivă ca variabile automate.

Declararea variabilelor registru se face cu ajutorul cuvântului cheie **register**:

register tip variabilă;

Alocarea este valabilă numai în funcția în care au fost declarate.

2.3. Exemplu de program modularizat

Următorul program calculează inversa și determinantul unei matrice pătrate cu elemente double. Programul a fost modularizat astfel:

Fișierul L5Ex1_1.cpp – conține funcția de citire a dimensiunilor și elementelor unei matrice și funcția de afișare a unei matrice:

```
/* Afișarea si citirea unei matrice de n*m elemente de tip  
double */  
  
#include <stdio.h>  
#include <conio.h>  
  
#define NMAX 10  
  
void afisare(int n,int m,double a[NMAX][NMAX],char ch)  
{  
    int i,j;  
    printf("\n MATRICEA %c\n",ch);  
    for(i=0;i<n;i++)  
        { for(j=0;j<m;j++)  
            printf("%8.2lf ",a[i][j]);  
            printf("\n");  
        }  
    }  
  
void citire_matrice(int *n,int *m,double a[NMAX][NMAX])  
{  
    int i,j;  
    printf("\nIntroduceti nr.linii n=");scanf("%d",n);  
    printf("\nIntroduceti nr.coloane m=");scanf("%d",m);  
    printf("\nIntroduceti elementele matricei\n");  
    for (i=0;i<*n;i++)  
        for(j=0;j<*m;j++)
```

```

        {
            printf("a[%d,%d]=",i,j);scanf("%lf",&a[i][j]);
        }
    printf("\n");
}

```

Fișierul L5Ex1_2.cpp – conține funcția de înmulțire a două matrice;

```

/* Functia calculeaza produsul matricelor a[n][m]
   si b[m][p] de tip double rezultand matricea c[n][p] */
#define NMAX 10

void produs(int n,int m,int p,double a[NMAX][NMAX],
            double b[NMAX][NMAX],double c[NMAX][NMAX])
/* Calculul produsului c=a*b) */
{
    int i,j,k;
    double s;
    for(i=0;i<n;i++)
        for(j=0;j<p;j++)
        {
            s=0.0;
            for(k=0;k<m;k++)
                s=s+a[i][k]*b[k][j];
            c[i][j]=s;
        };
}

```

Fișierul L5Ex1_3.cpp – conține funcția de calcul a inversei unei matrice pătrate și a determinatului afișat:

```

/*Program de calcul a inversei unei matrice si a determinantului
atasat */

#include <math.h>

#define NMAX 10

```

```

void invers(int n,double a[NMAX][NMAX],double eps,
           double b[NMAX][NMAX],double *det_a,
           int *err)
{
    int i,j,k,pozmax;
    double amax,aux;
    /* Initializarea matricei b cu matricea unitatwe */
    for(i=0;i<n;i++)
        for(j=0;j<n;j++)
            if(i==j) b[i][j]=1.0;
            else b[i][j]=0.0;
    /* Initializarea determinantului */
    *det_a=1.0;
    /* Se face 0sub diagonala principala si 1 pe ea */
    k=0; /*k=nr.liniei */
    *err=0;
    while((k<n) && (*err==0))
    {
        /*Calcul element pivot*/
        amax=fabs(a[k][k]);pozmax=k;

        for(i=k+1;i<n;i++)
            if (fabs(a[i][k]) >amax) {
                amax=fabs(a[i][k]);
                pozmax=i;
            };

        /*Interschimbarea liniei k cu pozmax in matr. a si b */
        if( k!=pozmax) {
            for(j=0;j<n;j++)
            {
                aux=a[k][j];
                a[k][j]=a[pozmax][j];
                a[pozmax][j]=aux;
                aux=b[k][j];
                b[k][j]=b[pozmax][j];
                b[pozmax][j]=aux;
            };
            *det_a=-*det_a;
        };
    };
}

```



```

if( fabs(a[k][k]) <eps) *err=1;
else {
    *det_a=*det_a*a[k][k];
    aux=a[k][k];
    for(j=0;j<n;j++)
    {
        a[k][j]=a[k][j] / aux;
        b[k][j]=b[k][j] / aux;
    };
    for(i=0;i<n;i++)
    if(i!=k) {
        aux=a[i][k];
        for(j=0;j<n;j++)
        {
            a[i][j]=a[i][j]-a[k][j]*aux;
            b[i][j]=b[i][j]-b[k][j]*aux;
        }
    }
    k++;
}
}

```

Fișierul L5Ex1_4.cpp – conține funcția main;

```

/*Program de calcul a inversei unei matrice si a
determinantului atasat */
#include <stdio.h>
#include <conio.h>

#define NMAX 10
void citire_matrice(int *n,int *m,double a[NMAX][NMAX]);
void afisare(int n,int m,double a[NMAX][NMAX],char ch);
void produs(int n,int m,int p,double a[NMAX][NMAX],
            double b[NMAX][NMAX],double c[NMAX][NMAX]);
void invers(int n,double a[NMAX][NMAX],double eps,
            double b[NMAX][NMAX],double *det_a,int *err);

```

```

void main()
{
    int i,j,n,m,err;
    double eps,det_a,a[NMAX][NMAX],a1[NMAX][NMAX],
        b[NMAX][NMAX],c[NMAX][NMAX];

    clrscr;
    citire_matrice(&n,&m,a);
    afisare(n,m,a,'A');
    getch();
    for(i=0;i<n;i++)
        for(j=0;j<n;j++)
            a1[i][j]=a[i][j];
    eps=1.0e-6;
    invers(n,a1,eps,b,&det_a,&err);
    if(err==1) printf("\nMATRICEA A ESTE SINGULARA");
    else { printf("\nMATRICEA INVERSA B=A^(-1)\n");
        afisare(n,n,b,'B');
        printf("\nDETERMINANTUL MATRICEI A ESTE
            %8.4lf",det_a);
        produs(n,n,n,a,b,c);
        printf("\nVERIFICARE C=A*B REZULTA MATRICEA
            UNITATE!");
        afisare(n,n,c,'C');
        getch();
    }
}

```

Fişierul L5Ex1_5.cpp – conţine construcţiile de includere a fişierelor de mai sus.

```

#include "d:\iosif\limbaj_C\L5Ex1_1.cpp"
#include "d:\iosif\limbaj_c\L5Ex1_2.cpp"
#include "d:\iosif\limbaj_c\L5Ex1_3.cpp"
#include "d:\iosif\limbaj_c\L5Ex1_4.cpp"

```

3. Mersul lucrării

3.1. Se va compila, executa și analiza programul dat ca exemplu mai sus.

3.2. Se vor compila separat modulele programului exemplificat și apoi cu link-editorul se va obține programul executabil.

Se va construi un fișier de tip Project cu modulele programului exemplificat.

În continuare se va scrie câte un program modularizat pentru rezolvarea următoarelor probleme:

3.3. Dându-se forma postfixată a unei expresii aritmetice care conține numai numere întregi și operatori $+$, $-$, $*$, $/$, să se scrie un program pentru evaluarea sa.

3.4. Să se scrie un program pentru calculul c.m.m.d.c. și a c.m.m.m.c a două polinoame.

3.5. Să se implementeze noțiunea de mulțime și operațiile permise asupra sa.

3.6. Să se implementeze un editor de texte care să permită câteva operații definite de Dvs. (inserarea unui text, ștergere, modificare etc. ...)

POINTERI

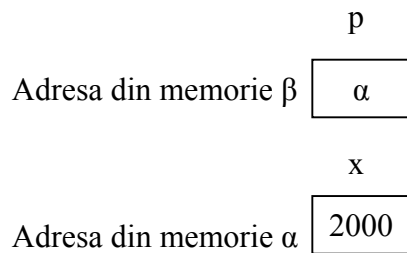
1. Conținutul lucrării

În lucrare se prezintă tipul pointer, operațiile permise asupra pointerilor, modul de alocare și eliberare dinamică a memoriei.

2. Considerații teoretice

2.1. Tipul pointer

Un pointer este o variabilă care are ca valori adrese. Dacă pointerul p are ca valoare adresa de memorie a variabilei x , se spune că p pointează spre x .



Un pointer este legat de un tip. Dacă x este de tipul `int`, pointerul p este legat de tipul `int`.

Declararea unui pointer se face la fel ca declararea unei variabile, cu deosebirea că numele pointerului este precedat de caracterul `*`:

tip *nume;

Exemplu:

int *p;

Adresa unei variabile se obține cu ajutorul operatorului unar **&**, numite **operator de referențiere**.

Exemplu: Fie declarațiile:

```
int x;  
int *p;
```

Atunci **p=&x;** are ca efect atribuirea ca valoare pentru p a adresei variabilei x. În desenul de mai sus, variabila x fiind localizată la adresa **a**, valoarea lui p va fi **a**.

Furnizarea valorii din zona de memorie a cărei adresă este conținută în p se face cu ajutorul operatorului unar *****, numit **operator de dereferențiere**.

Exemplu:

a) instrucțiunea **x=y** este echivalentă cu una din secvențele:

```
p=&x;      sau   p=&y;  
*p=y;      x=*p;
```

b) instrucțiunea **x++** este echivalentă cu secvența:

```
p=&x;  
(*p)++;
```

În aceste exemple p trebuia să fie legat de tipul lui x,y. De exemplu:

```
int x,y;  
int *p;
```

Există cazuri când un pointer trebuie să nu fie legat de un tip de date. În acest caz, se folosește declarația următoare:

```
void *nume;
```

În acest caz, dacă avem declarațiile:

```
int x;  
float y;
```

void *p;

atunci este corectă oricare din instrucțiunile:

p=&x;
p=&y;

Însă este necesară folosirea expresiilor de tip cast, pentru a preciza tipul datei spre care pointează p:

(tip *)p

Observație: este necesară cunoașterea în fiecare moment a tipului valorii ce se găsește la adresa atribuită pointerului de tip void *. Neținând seama de acest lucru se ajunge la erori.

Exemplu de utilizare a unui pointer de tip void:

int x;
void *p;

Instrucțiunea **x=10** este echivalentă cu secvența :

p=&x;
***(int *)p=10;**

În esență, dacă **p** este pointer declarat ca **void *p**, nu poate fi folosită dereferențierea ***p** fără a preciza tipul datei referite printr-o expresie de tipul cast.

2.2. Legătura dintre pointeri și tablouri

Numele unui tablou are drept valoare adresa primului său element. Ca urmare, se spune că numele unui tablou este un pointer constant, neputând fi modificat în timpul execuției.

Exemplu:

```

int tab[100];
int *p;
int x;
...
p=t; /* p primește ca valoare adresa elementului tab[0] */
...

```

În acest exemplu, atribuirea **x=tab[0]** este echivalentă cu

```
x=*p;
```

Ca urmare a celor prezentate, rezultă că dacă un parametru efectiv este un tablou unidimensional, atunci parametrul formal corespunzător poate fi declarat în două moduri:

a) ca tablou: **tip nume_parametru_formal[];**

b) ca pointer: **tip *nume_parametru_formal;**

Cele două declarații ale parametrului formal sunt echivalente, fiind corectă utilizarea în corpul funcției a construcției de variabilă indexată:

nume_parametru_formal [indice];

Acest lucru este ilustrat în exemplul de mai jos, care are drept scop găsirea maximului și minimului dintre elementele unui șir.

```
/* Programul L6Ex1.cpp */
```

```
/* Programul exemplifica transmiterea parametrului formal
   tablou prin pointer */
```

```
#include <stdio.h>
```

```
#include <conio.h>
```

```
void Max_min1(int n,int a[],int *max,int* min)
```

```
{
    int i;
    *max=a[0];
    *min=a[0];
    for (i=1;i<n;i++)
    {
        if (a[i]>*max) *max=a[i];
        else if (a[i]< *min) *min=a[i];
    }
}
```

```

void Max_min2(int n,int *a,int *max,int *min)
{
    int i;
    *max=a[0];
    *min=a[0];
    for (i=1;i<n;i++)
    {
        if (a[i]>*max) *max=a[i];
        else if (a[i]< *min) *min=a[i];
    }
}
void main(void)
{
    int i,n,maxim,minim;
    int x[100];
    /* Introducerea datelor */
    printf("\nNumarul elementelor tabloului n=");
    scanf("%d",&n);
    for(i=0;i<n;i++)
    {
        printf("\nx[%d]=",i);
        scanf("%d",&x[i]);
    };
    /* Apelul primei proceduri */
    Max_min1(n,x,&maxim,&minim);
    printf("\nLa apelul functiei Max_min1 rezulta:\n
           maximul=%d      minimul=%d\n",maxim,minim);
    /* Apelul celei de a doua proceduri */
    Max_min2(n,x,&maxim,&minim);
    printf("\nLa apelul functiei Max_min2 rezulta:\n
           maximul=%d  minimul=%d\n",maxim,minim);
    printf("\nApasati o tasta!\n");
    getch();
}

```

2.3. Operații asupra pointerilor

Asupra pointerilor sunt permise următoarele operații:

- a) Incrementare/decrementare cu 1. În acest caz valoarea pointerului este incrementată/decrementată cu numărul de octeți necesari pentru a păstra o dată de tipul de care este legat pointerul.
Operatorii folosiți sunt ++ și --.

De exemplu:

```
int tab[100];
int *p;
.....
p=&tab[10];
p++; /* Valoarea lui p este incrementată cu 2, având adresa
elementului tab[11]*/
```

- b) Adunarea și scăderea unui întreg dintr-un pointer.

Operația $p \pm n$ are drept efect creșterea, respectiv scăderea din valoarea p a n *numărul de octeți necesari pentru a păstra o dată de tipul de care este legat pointerul.

Pentru exemplul de mai sus, dacă x este de tipul **int**, atunci:

```
x=tab[i];
```

este echivalentă cu:

```
x=*(tab+i);
```

- c) Diferența a doi pointeri.

Dacă 2 pointeri p și q pointează spre elementele i și j ale aceluiași tablou ($j > i$), adică $p = \&tab[i]$ și $q = \&tab[j]$, atunci $q - p = (j - i) * \text{numărul de octeți}$ necesari pentru a păstra o dată de tipul de bază al tabloului.

- d) Compararea a doi pointeri

Doi pointeri care pointează spre elementele aceluiași tablou pot fi comparați folosind operatorii de relație și de egalitate:

< <= > >= == !=

Mai jos este prezentat programul de la paragraful precedent, folosind operații asupra pointerilor.

```

/* Programul L6Ex2.cpp */

/* Programul exemplifica folosirea operatiilor
asupra pointerilor */
#include <stdio.h>
#include <conio.h>

void Max_min1(int n,int a[],int *max,int* min)
{
    int i;
    *max=a[0];
    *min=a[0];
    for (i=1;i<n;i++)
    {
        if (a[i]>*max) *max=a[i];
        else if (a[i]< *min) *min=a[i];
    }
}

void Max_min2(int n,int *a,int *max,int *min)
{
    int i;
    *max=*a;
    *min=*a;
    for (i=1;i<n;i++)
    {
        if (*(a+i)>*max) *max=*(a+i);
        else if (*(a+i)< *min) *min=*(a+i);
    }
}

void main(void)
{
    int i,n,maxim,minim;
    int x[100];
    /* Introducerea datelor */
    printf("\nNumarul elementelor tabloului n=");
    scanf("%d",&n);
    for(i=0;i<n;i++)

```

```

    {
        printf("\nx[%d]=",i);
        scanf("%d",&x[i]);
    };
/* Apelul primei proceduri */
Max_min1(n,x,&maxim,&minim);
printf("\nLa apelul functiei Max_min1 rezulta:\n
        maximul=%d minimul=%d\n",maxim,minim);
/* Apelul celei de a doua proceduri */
Max_min2(n,x,&maxim,&minim);
printf("\nLa apelul functiei Max_min2 rezulta:\n
        maximul=%d minimul=%d\n",maxim,minim);
printf("\nApasati o tasta!\n");
getch();
}

```

2.4. Alocarea/eliberarea dinamică a memoriei heap

Alocarea memoriei pentru variabilele globale și statice este statică, adică alocarea rămâne până la terminarea programului.

Alocarea memoriei pentru variabilele automate este dinamică, în sensul că stiva este “curățată” la terminarea funcției.

Memoria heap este o zonă de memorie dinamică, specială, distinctă de stivă. Ea poate fi gestionată prin funcții, care au prototipurile în fișierul `alloc.h` și `stdlib.h`.

Alocarea unei zone de memorie heap se poate realiza cu ajutorul funcțiilor de prototip:

```

void *malloc (unsigned n);
void *calloc(unsigned nr_elem, unsigned dim);

```

Funcția **malloc** alocă în heap o zonă contiguă de **n octeți**, iar funcția **calloc** o zonă contiguă de **nr_elem * dim** în octeți.

Funcțiile returnează:

- în caz de succes, adresa de început a zonei alocate (pointerul fiind de tip void, este necesară conversia spre tipul dorit);
- în caz de insucces, returnează zero (pointerul NULL);

Eliberarea unei zone alocate cu malloc sau calloc se face cu ajutorul funcției de prototip:

void free (void *p);

Observație: Pentru alocări de blocuri mai mari de 64 kocteți, este necesară utilizarea pointerilor de tipul far. În acest caz, funcțiile de mai sus au prototipurile:

void far *farmalloc (unsigned long n);
void far *farcalloc(unsigned long nr_elemente, unsigned long
dim);
void farfree (void far *p);

Mai jos se prezintă un exemplu de utilizare a funcțiilor prezentate în acest paragraf.

/* Programul L6Ex3.cpp */

```
#include <stdio.h>  
#include <alloc.h>  
#include <process.h>  
#include <conio.h>  
void main(void)  
{  
    char *str1,*str2;  
  
    /* Aloca memorie pentru primul sir de caractere */  
    if ((str1 = (char *) malloc(100)) == NULL)  
    {  
        printf("Memorie insuficienta\n");  
        exit(1);  
    }  
    printf("\nIntroduceti primul sir de caractere terminat cu  
        ENTER\n");  
    gets(str1);  
    printf("\nSirul de caractere introdus este\n  %s\n",str1);
```

```

/* Aloca memorie pentru al doilea sir de caractere */
if ((str2 = (char *) calloc(100,sizeof(char))) == NULL)
{
    printf("Memorie insuficienta\n");
    exit(2);
}
printf("\nIntroduceti al doilea sir de caractere terminat cu
        ENTER\n");
gets(str2);
printf("\nSirul de caractere introdus este\n  %s\n",str2);
printf("\nApasati o tasta\n");
getch();
/* Eliberarea memoriei */
free(str1);
free(str2);
}

```

2.5. Folosirea ca parametru a unei funcții

Numele unei funcții este un pointer spre funcția respectivă. De aceea, numele unei funcții poate fi folosit ca parametru efectiv la apelul unei funcții.

Fie **f** o funcție care va fi transmisă ca parametru efectiv, având antetul:

```
tipf f(lista_parametrilor_formali_f);
```

În acest caz, parametrul formal al unei funcții **g** care va fi apelată cu parametrul efectiv **f**, este prezentat în antetul funcției **g**:

```
tipg g(..., tipf(*p)(lista_parametrilor_formali_f), ...)
```

Apelul se va face astfel:

```
g(...,f,..);
```

Programul L6Ex4.cpp prezintă un exemplu în acest sens. Este vorba de integrarea unei funcții prin metoda trapezului.

$$\int_a^b f(x)dx = h \left(\frac{f(a) + f(b)}{2} + \sum_{i=1}^{n-1} f(a + i * h) \right)$$

unde, n este numărul de subintervale în care s-a împărțit intervalul [a,b], dimensiunea unui subinterval fiind h.

/* Programul L6Ex4.cpp */

**/* Programul exemplifica modul de folosire
a unei functii ca parametru */**

```
#include <stdio.h>
#include <conio.h>
#include <math.h>
double f(double x)
{
    return (3*x*x +1);
}
double integrala(double a,double b,int n,double(*p)(double x))
/* Calculul integralei prin metoda trapezelor */
{
    int i;
    double h,s;
    h=(b-a)/n;
    s=((*p)(a)+(*p)(b))/2.0;
    for(i=1;i<n;i++)
        s=s+(*p)(a+i*h);
    s=s*h;
    return s;
}
void main()
{
    double a,b;
    int n;
    char ch;
    /* Citirea intervalului de integrare */
    printf("\na=");scanf("%lf",&a);
    printf("\nb=");scanf("%lf",&b);
    ch='D';
```

```

while (ch=='D' || ch=='d')
{
    printf("\nn=");scanf("%d",&n);
    printf("\nPentru n=%d Valoarea integralei este %lf",n,
        integrala(a,b,n,f));
    printf("\nApasati o tasta\n");getch();
    printf("\nIntroduceti alt n? DA=D/d NU=alt caracter ");
    ch=getch();
}
}

```

3. Mersul lucrării

3.1 Se vor executa exemplele din lucrare. Se vor analiza următoarele lucruri:

- folosirea unei variabile indexate, atunci când numele tabloului a fost definit ca pointer în antetul funcției (L6Ex1.cpp);
- operațiile permise asupra pointerilor. Ce avantaj prezintă înlocuirea unei variabile cu indici cu o expresie cu pointeri? (L6Ex2.cpp);
- cum se alocă în memoria heap spațiul de memorie pentru variabile dinamice? (L6Ex3.cpp).
- care este avantajul transmiterii funcțiilor ca parametru efectiv (L6Ex4.cpp).

În continuare se vor scrie programele pentru rezolvarea următoarelor probleme:

3.2. Folosind numai pointeri și expresii cu pointeri se vor scrie funcții de citire, afișare și înmulțire a două matrice.

3.3. Folosind numai pointeri și expresii cu pointeri se vor scrie funcții de sortare a unui vector cu elemente reale.

3.4. Folosind numai pointeri și expresii cu pointeri se va scrie o funcție de interclasare a doi vectori, care conțin elemente de tip real ordonate crescător.

3.5. Să se scrie o funcție care sortează în ordine crescătoare n șiruri de caractere.

3.6. Să se scrie o funcție care determină rădăcina unei funcții $f(x)$, în intervalul $[a,b]$, știind că admite o singură rădăcină în acest interval. Funcția f va fi transmisă ca parametru efectiv.

RECURSIVITATE

1. Conținutul lucrării

În lucrare este prezentată noțiunea de recursivitate, avantajele și dezavantajele funcțiilor recursive în raport cu cele nerecursive, pe baza unor exemple simple.

2. Considerații teoretice

2.1. Mecanismul recursivității

Un obiect este recursiv dacă este definit prin el însuși. O funcție este recursivă dacă ea se autoapelează.

Recursivitatea poate fi:

- directă - când funcția conține un apel direct la ea însăși;
- indirectă - când funcția conține un apel al altei funcții, care la rândul său o apelează pe prima.

La fiecare apel al unei funcții, parametrii și variabilele automate ale ei se alocă pe stivă într-o zonă independentă. Acest lucru se întâmplă la fiecare apel sau autoapel al funcției. De aceea datele amintite au valori distincte la fiecare reapelare. Variabilele statice și cele globale ocupă tot timpul aceeași locație de memorie. Ca urmare, orice modificare asupra lor se face numai la adresa fixată în memorie, deci ele își păstrează valoarea de la un reapel la altul.

Revenirea dintr-o funcție se face în punctul următor celui din care

s-a făcut apelul. Adresa de revenire se păstrează tot în stivă. La revenire, stiva se reface la starea ei dinaintea apelului, deci variabilele automate și parametrii vor reveni la valorile lor dinaintea reapelului respectiv.

O problemă importantă este stoparea autoapelului. De aceea trebuie să existe o condiție de terminare, fără de care un apel recursiv ar conduce la o buclă infinită. În aplicațiile practice este necesar nu numai ca adâncimea recursivității să fie finită, ci să fie relativ mică, deoarece fiecare apel recursiv necesită alocarea pe stivă a zonei de memorie pentru:

- parametrii funcției;
- variabilele automate locale funcției;
- adresa de return (revenire în punctul de apel).

Ca urmare, stiva poate crește foarte mult și repede se ajunge la ocuparea întregului spațiu de memorie alocat ei.

Un exemplu clasic de proces recursiv este calculul factorialului definit astfel:

$$fact(n) = \begin{cases} 1 & \text{daca } n = 0 \\ n * fact(n-1) & \text{daca } n > 0 \end{cases}$$

Se observă că în definiția funcției fact există o parte care nu se definește prin ea însăși și anume $fact(n)=1$ dacă $n=0$.

În limbajul C/C++, codul funcției corespunzătoare este următoarea:

```
double fact(int n)  
{  
    if (n==0) return 1.0;  
    else return n*fact(n-1)  
}
```

Recursivitatea liniară se caracterizează prin faptul că nu pot apărea pe ramuri diferite ale execuției programului mai multe apeluri recursive, adică pe un anumit nivel apare doar un singur apel recursiv.

Recursivitatea liniară întotdeauna poate fi transformată în iterație, ducând la economie de memorie și timp de calcul (se elimină operațiile de salvare de context la autoapelurile funcției).

Avantajul principal al recursivității este scrierea mai compactă și mai clară a funcțiilor care exprimă procese de calcul recursive. În această clasă

de procese intră cele generate de metodele de căutare cu revenire (“backtracking”) și metodele de divizare (“divide et impera”).

2.2. Exemple

2.2.1. Citirea a n cuvinte (șiruri de caractere), fiecare terminat cu spațiu și tipărirea lor în oglindă.

/* Programul L7Ex1.cpp */

```
#include <stdio.h>  
#include <conio.h>
```

```
/* Programul citeste n cuvinte separate cu spatiu;  
(dupa ultimul cuvant va exista spatiu si <ENTER>)  
si le afiseaza "in oglinda" */
```

```
void revers(void)  
{  
    char c;  
    scanf("%c",&c);  
    if (c!='\40') {printf("%c",c);revers();}  
    printf("%c",c);  
}  
  
void main(void)  
{  
    int n,i;  
    printf("\nNumarul de cuvinte=");  
    scanf("%d",&n);  
    for(i=1;i<=n;++i)  
        {  
            revers();  
            printf("\n");  
        };  
    printf("\nPROGRAMUL S-A TERMINAT!!!\n");  
    getch();  
}
```

Funcția revers citește câte un caracter pe care îl afișează până la întâlnirea spațiului (terminatorul șirului de caractere). Fiecare autoapel conduce la păstrarea în stivă a variabilei locale c. Apariția spațiului conduce la terminarea apelurilor recursive ale funcției, urmând scrierea spațiului și a caracterelor în ordinea inversă introducerii lor.

2.2.2. Determinarea termenului minim al unui șir de n întregi.

```
/* Programul L7Ex2.cpp */
```

```
#include <stdio.h>
```

```
#include <conio.h>
```

```
/* Programul calculeaza minimul dintr-un sir  
    cu termeni intregi */
```

```
#define NMAX 100
```

```
#define MAXIM 32767
```

```
int sir[NMAX];
```

```
int minim(int x,int y)
```

```
{  
    if (x<=y) return x;  
    else return y;  
}
```

```
int termen_minim(int dim_sir)
```

```
{  
    if (dim_sir>=0) return minim(sir[dim_sir],termen_minim(dim_sir-1));  
    else return MAXIM;  
}
```

```
void main(void)
```

```
{  
    int i,n;  
    printf("\nIntroduceti nr de termeni ai sirului n=");  
    scanf("%d",&n);  
    printf("\nIntroduceti valorile termenilor\n");  
    for(i=0;i<n;++i)  
    {  
        printf("sir[%d]=",i);
```

```

    scanf("%d",&sir[i]);
};
printf("\nSIRUL INTRODUS\n");
for(i=0;i<n;++i)
{
    printf("%6d",sir[i]);
    if ((i+1) % 10 == 0) printf("\n");
};
printf("\nCel mai mic termen este %d\n",termen_minim(n-1));
printf("\nApasati o tasta!");
getch();
}

```

2.2.3. Varianta recursivă și nerecursivă a găsirii valorii celui de al n-lea termen al șirului lui Fibonacci.

Șirul lui Fibonacci este definit astfel:

$$\begin{aligned}
 &\text{Fib}(0)=0; \text{ Fib}(1)=1; \\
 &\text{Fib}(n)=\text{Fib}(n-1)+\text{Fib}(n-2) \quad \text{pentru } n \geq 2
 \end{aligned}$$

/* Programul L7Ex3.cpp */

```

#include <stdio.h>
#include <conio.h>

```

/* Numerele lui Fibonacci */

```

int fib1(int n)
/* VARIANTA RECURSIVA */
{
    if (n==0) return 0;
    else if (n==1) return 1;
        else return (fib1(n-1)+fib1(n-2));
}

```

```

int fib2(int n)
/* VARIANTA NERECURSIVA */
{
    int i,x,y,z;
    if (n==0) return 0;

```

```

else if (n==1) return 1;
    else {
        x=1;y=0;
        for(i=2;i<=n;++i)
        {
            z=x;x=x+y;y=z;
        };
        return x;
    }
}

void main(void)
{
    int n;
    char ch;
    ch='D';
    while ((ch=='d')|| (ch=='D'))
    {
        printf("\nIntroduceti n=");
        scanf("%d",&n);
        printf("\nCALCUL RECURSIV:  fib(%d)=%d\n",n,fib1(n));
        printf("\nCALCUL NERECURSIV: fib(%d)=%d\n",n,fib2(n));
        printf("\nDoriti sa continuati ? Da=D/d");
        ch=getch();
    }
}

```

Apelul recursiv conduce la creșterea rapidă a stivei, de aceea este preferabilă implementarea nerecursivă.

3. Mersul lucrării

3.1. Se vor analiza și executa programele din exemplele de mai sus. Pentru un caz concret, se va trasa starea stivei, urmărindu-se creșterea pe măsura autoapelării funcției și scăderea ei la revenirea din autoapel.

3.2. Să se scrie o funcție recursivă și una nerecursivă pentru calculul valorii polinoamelor Hermite $H(x)$ definite astfel:

$$H_0(x)=1; \quad H_1(x)=2x; \quad H_n(x)=2nH_{n-1}(x)-2(n-1)H_{n-2}(x),$$

pentru $n \geq 2$

3.3. Problema turnurilor din Hanoi

Se consideră trei tije verticale A,B,C și n discuri de diametre diferite. Inițial toate discurile sunt puse în tija A, în ordinea descrescătoare a diametrului (discul cel mai mare la bază, iar cel mai mic în vârf). Se cere să se mute discurile de pe tija A pe tija C folosind tija B ca intermediar, folosind condițiile:

- la o manevră se mută un singur disc și anume cel din vârful unei tije;
- nu se poate pune un disc de diametru mai mare peste unul de diametru mai mic;
- în final, pe tija C, discurile trebuie să fie în aceeași ordine ca în starea inițială de pe tija A.

3.4.Să se scrie un program recursiv care citește n cuvinte și le afișează în ordinea inversă a introducerii lor.

3.5.Să se scrie un program recursiv de generare a produsului cartezian a n mulțimi.

3.6.Să se scrie un program de generare recursivă a submulțimilor de k elemente ale mulțimii A cu n elemente (combinațiile de n elemente luate câte k).

3.7. Să se scrie un program de rezolvare a problemei celor 8 regine (determinarea tuturor așezărilor pe tabla de șah a celor 8 regine astfel încât să nu se atace).

3.8.Să se genereze recursiv permutările mulțimii A de n elemente.

3.9.Se consideră o bară de lungime m și n repere de lungimi $l_1, l_2, \dots, l_n$. Din bară trebuie tăiate bucăți de lungimea reperelor date, astfel încât să rezulte din fiecare reper cel puțin o bucată și pierderile să fie minime.

3.10. Funcția lui Ackermann. Să se scrie programul recursiv care calculează funcția lui Ackermann definită astfel:

$\text{Ack}(0,n)=n+1$	pentru $n \in \mathbb{N}$
$\text{Ack}(m,0)=\text{Ack}(m-1,1)$	pentru $m \in \mathbb{N}^*$
$\text{Ack}(m,n)=\text{Ack}(m-1,\text{Ack}(m,n-1))$	pentru $m,n \in \mathbb{N}^*$

ȘIRURI DE CARACTERE

1. Conținutul lucrării

În lucrare se prezintă modul de reprezentare în memorie a unui șir de caractere și unele funcții standard de prelucrare a șirurilor de caracter.

2. Considerații teoretice

2.1. Reprezentarea în memorie a unui șir de caractere

Un șir de caractere este păstrat într-un tablou unidimensional de tip char. Fiecare caracter se păstrează într-un octet prin codul ASCII al său. Ultimul caracter al șirului, deci terminatorul șirului, este caracterul NULL ('0').

Numele tabloului care păstrează șirul de caractere este un pointer constant spre șirul de caractere.

Exemplu:

```
char sir []="SIR DE CARACTERE";
```

În memorie reprezentarea sa va fi (în hexazecimal):

sir	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
	53	49	52	20	44	45	20	43	41	52	41	43	54	45	52	45	00

sir are ca valoare adresa zonei de memorie care conține șirul.

Avem următoarele relații:

`sir[i]` unde $i \in [0,16]$ – reprezintă codul ASCII al celui de al i-lea caracter din șirul de caractere;

`sir + i` unde $i \in [0,16]$, reprezintă adresa celui de al i-lea caracter din șirul de caractere;

`*(sir+i)` are același efect ca `sir[i]`.

Tot ce s-a explicat mai sus, rămâne valabil și în cazul declarării în felul următor:

```
char *const sir="SIR DE CARACTERE";
```

Declararea unui tablou de șiruri de caractere se poate face astfel:

```
char *tab[]={sir_0,sir_1,...,sir_n};
```

În acest caz, `tab[i]`, pentru $i \in [0,n]$, este un pointer spre șirul de caractere "`sir_i`".

În cazul apelului

```
printf("%s\n", tab[i]);
```

se va afișa textul `sir_i`.

2.2. Funcții standard de prelucrare a șirurilor de caractere

Funcțiile standard de citire/scriere a șirurilor de caractere:

- `gets/puts`;
- `scanf/printf`;
- `sscanf/sprintf`

au fost prezentate în lucrarea de laborator nr. 1.

În continuare sunt prezentate câteva funcții de prelucrare a șirurilor de caractere, al căror prototip se găsește în fișierul **string.h**

2.2.1. Lungimea unui șir de caractere

Lungimea unui șir de caractere, fără a fi luat în considerare caracterul `'\0'`, este returnat de funcția **strlen**, care are prototipul:

```
unsigned strlen (const char *s);
```

Exemplu:

```
/* Programul L8Ex1.cpp */

/* Programul exemplifica utilizarea functiei strlen */
#include <stdio.h>
#include <conio.h>
#include <string.h>
#define alfa "Apasati o tasta!"

void main(void)
{
    char sir1[]="SIR DE CARACTERE";
    char *sir2="SIR DE CARACTERE";
    int n1,n2,n3;
    n1=strlen(sir1);
    n2=strlen(sir2);
    n3=strlen("SIR DE CARACTERE");
    /* Atat n1,cat si n2 si n3 au ca valoare 16 */
    printf("\n n1=%d n2=%d n3=%d\n",n1,n2,n3);
    printf("%s\n",alfa);
    getch();
}
```

2.2..2. Copierea unui șir de caractere

Copierea unui șir de caractere dintr-o zonă de memorie de adresă **sursă** într-o altă zonă de memorie de adresă **dest** se face cu ajutorul funcției *strcpy*, al cărei prototip este:

char *strcpy (char *dest, const char *sursă);

Se menționează că are loc copierea inclusiv a caracterului `NULL('0')`.

Funcția returnează adresa unde a avut loc copierea, adică chiar destinația.

Pentru a copia cel mult n caractere, din zona de memorie de adresă **sursă** în zona de memorie de adresă **dest**, se va folosi funcția **strncpy**, al cărei prototip este următorul:

char *strncpy (char *dest, const char *sursă, unsigned n);

După ultimul caracter transferat, trebuie pus caracterul '\0'.

Evident că dacă $n >$ lungimea șirului de la adresa sursă, atunci are loc transferarea întregului șir de caractere.

Exemplu:

/* Programul L8Ex2.cpp */

/* Programul exemplifica utilizarea functiei strcpy */

#include <stdio.h>

#include <conio.h>

#include <string.h>

#define alfa "\nApasati o tasta!"

void main(void)

{

char sir1[]="SIR DE CARACTERE";

char *sir2="SIR DE CARACTERE";

char sir3[100],sir4[100],sir5[100];

strcpy(sir3,sir1);

printf("\n sir3 contine: %s\n",sir3);

strcpy(sir4,"Functii standard de prelucrare siruri de caractere");

printf("\n sir4 contine: %s\n",sir4);

strncpy(sir5,sir2,6);/* sir5 contine SIR DE */

sir5[6]='\0';

printf("\n sir5 contine: %s\n",sir5);

printf(alfa);

getch();

}

2.2.3. Concatenarea a două șiruri de caractere

Adăugarea șirului de caractere de la adresa **sursă** după ultimul caracter (cel care precede NULL) al șirului de caractere de la adresa **dest** se face cu ajutorul funcției **strcat**, care are prototipul:

```
char *strcat(char *dest, const char *sursă);
```

După șirul rezultat, se pune evident caracterul NULL ('\0'). Funcția returnează valoarea adresei destinație. Există posibilitatea de a prelua din șirul de caractere de la adresa sursă numai n caractere, cu ajutorul funcției **strncat**, care are prototipul:

```
char *strncat (char *dest, const char *sursa, unsigned n);
```

La sfârșit se pune automat caracterul NULL ('\0').

Dacă n > lungimea șirului de la adresa sursă, atunci funcția **strncat** are același efect ca și funcția **strcat**.

Exemplu:

```
/* Programul L8Ex3.cpp */  
  
/* Programul exemplifica utilizarea functiei strcat*/  
#include <stdio.h>  
#include <conio.h>  
#include <string.h>  
#define alfa "\nApasati o tasta!"  
  
void main(void)  
{  
    char sir1[100]="SIR1 DE CARACTERE";  
    char *sir2="<SIR2 DE CARACTERE";  
    char sir3[100];  
    strcpy(sir3,sir1);  
    strcat(sir1,sir2);  
    printf("\n sir1 contine: %s\n",sir1);  
    strncat(sir3,sir2,5);  
    /* Dupa ultimul caracter din sr3 se pune implicit '\0' */  
    for (int i=0;i<=strlen(sir3)+1;++i) printf("%x",sir3[i]);
```

```

printf("\n sir3 contine: %s\n",sir3);
printf(alfa);
getch();
}

```

2.2.4. Compararea a două șiruri de caractere

Compararea a două șiruri de caractere se face caracter cu caracter (pe baza codurilor ASCII) până când:

- s-a ajuns la un caracter i din primul șir care diferă de caracterul i din al doilea șir;
- s-a ajuns la sfârșitul unuia din șiruri sau a ambelor.

Compararea a două șiruri de caractere de la adresele **sir1** și respectiv **sir2** se poate face cu funcția de prototip:

```
int strcmp(const char *sir1,const char *sir2);
```

Funcția returnează:

- o valoare negativă dacă șirul de caractere de la adresa **sir1** este mai mic decât cel de la adresa **sir2**;
- zero dacă șirurile sunt egale;
- o valoare pozitivă, dacă șirul de la adresa **sir1** este mai mare decât cel de la adresa **sir2**.

Dacă dorim să se compare numai primele n caractere din cele două șiruri se folosește funcția de prototip:

```
int strncmp (const char *sir1, const char *sir2, unsigned n);
```

Dacă dorim să nu se facă distincție între literele mici și cele mari, atunci cele două funcții au drept corespondențe:

```
int stricmp (const char *sir1, const char *sir2);
int strnicmp (const char *sir1, const char *sir2, unsigned n);
```

Exemplu:

```
/* Programul L8Ex4.cpp */

/* Programul exemplifica utilizarea functiei strcmp*/
#include <stdio.h>
#include <conio.h>
#include <string.h>
#define alfa "\nApasati o tasta!"

void main(void)
{
    char sir1[100]="SIR DE CARACTERE";
    char *sir2="SIR de caractere";
    int i,j,k,l;
    i=strcmp(sir1,sir2);/* i<0 , rezulta sir1<sir2 */
    printf("\ni=%d\n",i);
    j=strncmp(sir1,sir2,3);/*j=0 ,rezulta ca primele 3 caractere din sir1 si
sir2 sunt egale */
    printf("\nj=%d\n",j);
    k=strcmp(sir1,sir2); /* k=0, rezulta ca cele 2 siruri sunt egale */
    printf("\nk=%d\n",k);
    l=strnicmp(sir1,"SIR de 10 caractere",6); /*l=0 */
    printf("\nl=%d\n",l);
    printf(alfa);
    getch();
}
```

3. Mersul lucrării

3.1. Se vor analiza și executa programele din lucrare.

3.2. Se va scrie o funcție care să realizeze extragerea dintr-un șir de caractere sursă a unui subșir specificat prin poziția în cadrul sursei și a numărului de caractere extrase.

3.3. Se va scrie o funcție pentru inserarea unui șir de caractere sursă într-un șir de caractere destinație, specificând poziția din care începe inserarea.

3.4. Se va scrie o funcție pentru ștergerea unui subșir dintr-un șir de caractere dat. Subșirul se va specifica prin poziție și număr de caractere.

3.5. Se va scrie o funcție pentru a verifica dacă un șir dat este subșir al unui alt șir de caractere. În caz afirmativ, se va specifica poziția pe care se regăsește pentru prima dată.

3.6. Să se scrie două funcții, una care convertește un număr întreg sau real într-un șir de caractere, iar cealaltă face operația inversă.

3.7. Să se scrie un program care citește n șiruri de caractere și afișează șirul cel mai lung și șirul cel mai mare alfanumeric.

TIPURILE DE DATE STRUCTURĂ, UNIUNE ȘI ENUMERARE

1. Conținutul lucrării

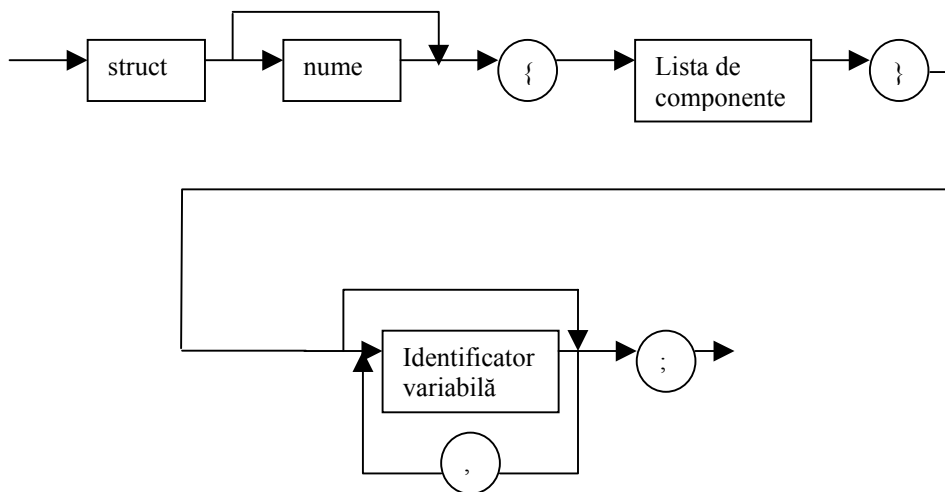
În lucrare sunt prezentate tipurile definite de utilizator structură, uniune și enumerare, accesul la componentele lor și asignarea de nume pentru aceste tipuri.

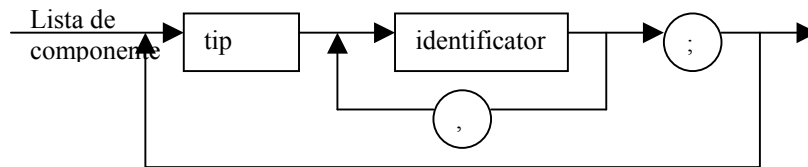
2. Considerații teoretice

2.1. Tipul de date "structură"

O **structură** conține mai multe componente de tipuri diferite (predefinite sau definite de utilizator), grupate conform unei ierarhii.

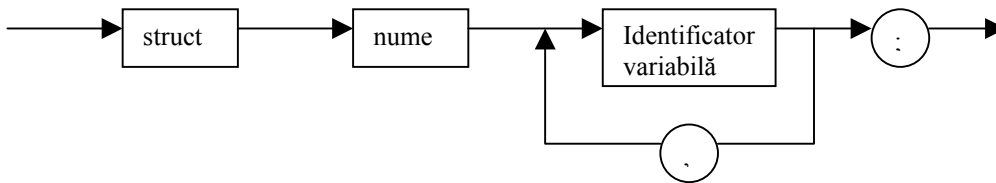
Declarația unei structuri se poate face astfel:





Observație: În aceasta declarație nu pot lipsi simultan "nume" și "identificator_variabilă".

O variabilă structură de tipul "nume" poate fi declarată și ulterior conform diagramei de mai jos, cu mențiunea că cuvântul cheie "struct" poate lipsi în C++:



Exemple echivalente:

a) struct material {
 long cod;
 char den [30];
 char um [10];
 real cantitate;
 real pret_unit;
 } stofa, hartie, motor;

b) struct material {
 long cod;
 char den [30];
 char um [10];
 real cantitate;
 real preț_unitar;
 };

struct material stofa, hartie, motor;

sau

material stofa, hartie, motor;

```
c) struct {  
    long cod;  
    char den [30]  
    char um [10];  
    real cantitate;  
    real pret_unitar;  
} stofa, hartie, motor;
```

Accesul la componentele unei structuri se poate face prin procedeul de calificare:

identificator_variabilă.identificator_câmp;

Exemplu: stofa.den
 hartie.cantitate

Procedeul de calificare pătrunde din aproape în aproape în ierarhia structură.

În limbajul C, transmiterea ca parametru a unei structuri la apelul unei funcții, se face numai prin adresa variabilei de tip structură. De exemplu:

void f (struct material *p, ...);

Apelul se va face prin:

f(&stofa, ...)

În funcție, selectarea unui câmp se face astfel:

(*p).den
(*p).cantitate

sau înlocuind **(*p)**. prin **p->**, ca mai jos:

p->den
p->cantitate

În limbajul C++, o structură poate fi transferată prin parametri în 3 moduri:

- direct: void f (material p, ..)
- pointer spre structură: void f (material *p, ...)
- referință la structură: void f (material &p, ...)

O structură de același tip se poate atribui direct una alteia:

```
material alfa, beta;
alfa=beta; {este corect}
```

2.2. Tipul de date "uniune"

În momente diferite ale execuției, se pot păstra în aceeași zonă de memorie date de tipuri diferite pentru economisirea memoriei sau din motive ale concepției programului. Acest lucru se face grupând toate datele care se alocă în aceeași zonă de memorie. Structura utilizator obținută se numește **uniune**. Sintaxa uniunii este identică cu cea a structurii, singura deosebire constând în înlocuirea cuvântului cheie "**struct**" cu "**union**" în diagramele de sintaxă de la punctul 2.1.

De menționat că zona de memorie rezervată are dimensiunea componentei care necesită cea mai multă memorie pentru reprezentare.

Accesul la componente se face identic ca la structură.

De menționat că programatorul trebuie să cunoască în fiecare moment care dată este reprezentată.

Exemplu:

```
union alfa
{
    char c[5]; /* reprezentare pe 5 octeți */
    int i;     /* reprezentare pe 2 octeți */
    long j;    /* reprezentare pe 4 octeți */
};

union alfa x;
strcpy(x.c, "ABCD");
```

Variabila x are reprezentarea în hexazecimal, astfel:

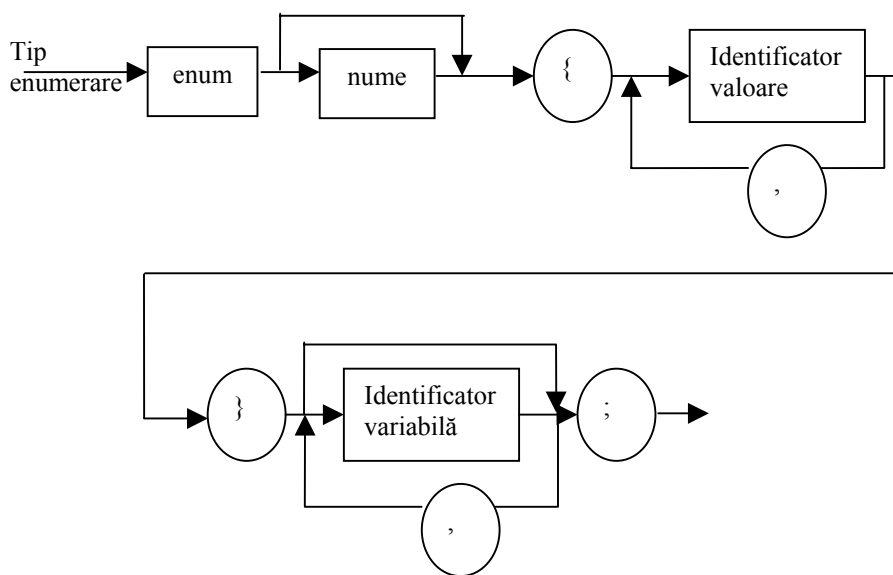
41	42	43	44	00
----	----	----	----	----

Daca se accesează componenta x.i, atunci aceasta va avea 4241 în hexazecimal, adică 16961 în zecimal.

În schimb, aceeași zonă de memorie interpretată ca x.j (long) va avea valoarea 44434241 în hexazecimal, adică 1.145.258.561 în zecimal

2.3. Tipul de date "enumerare"

Tipul **enumerare** permite programatorului de a folosi nume sugestive pentru valori numerice. Diagrama de sintaxă pentru tipul enumerare este asemănătoare cu tipurile structură și uniune. Deosebirea constă în faptul că lista de componente este formată numai din identificatori de valoare întregă 0,1,2,...:



Exemple echivalente:

a) enum săpt {luni, marți, miercuri, joi, vineri, sâmbătă, duminică};
enum săpt săpt_vacanță;

b) enum săpt {luni, marți, miercuri, joi, vineri, sâmbătă, duminică}
săpt_vacanță;

c) enum {luni, marți, miercuri, joi, vineri, sâmbătă, duminică} săpt_vacanță;

Atribuiți posibile:

săpt_vacanță=vineri;

Identificatorii luni, marți,..., au valorile 0,1,...

2.4. Declararea tipurilor de date prin nume simbolice

În limbajul C/C++ se poate atribui un nume simbolic unui tip predefinit sau unui tip utilizator. Diagrama de sintaxă pentru asignarea unui nume simbolic "nume_tip" unui "tip" predefinit sau utilizator este următoarea:



Exemplu:

a) typedef struct {
 int i;
 float j;
 double x;
} ALFA;

ALFA y, z;

b) typedef struct {
 float re;
 float im;
} COMPLEX;

COMPLEX x, y;

c) typedef union {
 char x[10];
 long cod;

```

        } BETA;
    BETA u, v;

d) typedef enum {false, true} BOOLEAN;
    BOOLEAN k, l;

```

2.5. Exemple de programe

Programul următor prezintă operații asupra numerelor complexe, folosind tipul structură. Sunt ilustrate toate posibilitățile de transmiterea a parametrilor de tip structură.

```

/*Programul L9Ex1.cpp */

#include <stdio.h>
#include <conio.h>
#include <process.h>
typedef struct {float re,im;}COMPLEX;

void aduna(COMPLEX *a,COMPLEX *b,COMPLEX *c)
/* transmiterea parametrilor prin pointeri */
{
    c->re=a->re+b->re;
    c->im=a->im+b->im;
};

void scade(COMPLEX a,COMPLEX b,COMPLEX *c)
/* transmiterea parametrilor prin valoare "posibil numai in C++"
   si a rezultatului prin pointer */
{
    c->re=a.re-b.re;
    c->im=a.im-b.im;
};

void produs(COMPLEX a,COMPLEX b,COMPLEX &c)
/*transmiterea parametrilor prin valoare "posibil numai in C++"
   si a rezultatului prin referinta */
{
    c.re=a.re*b.re-a.im*b.im;

```

```

    c.im=a.im*b.re+a.re*b.im;
};

void impartire(COMPLEX *a,COMPLEX *b,COMPLEX *c)
/*transmiterea parametrilor prin pointeri */
{
    float x;
    x=b->re*b->re+b->im*b->im;
    if (x==0) {
        printf("\nÎmpartire la zero!\n");
        exit(1);
    }
    else{
        c->re=(a->re*b->re+a->im*b->im)/x;
        c->im=(a->im*b->re-a->re*b->im)/x;
    }
};

void main(void)
/* Operații asupra numerelor complexe */

{
    COMPLEX a,b,c;
    char ch,op;
    ch='D';
    while ((ch=='D')|| (ch=='d'))
    {
        printf("\nIntroduceti primul număr complex\n");
        printf("a.re=");scanf("%f",&a.re);
        printf("a.im=");scanf("%f",&a.im);
        printf("\nIntroduceți al doilea număr complex\n");
        printf("b.re=");scanf("%f",&b.re);
        printf("b.im=");scanf("%f",&b.im);
        aduna(&a,&b,&c);
        printf("\n(%f+j*%f)+( %f+j*%f)=%f+j*%f\n",
            a.re,a.im,b.re,b.im,c.re,c.im);
        scade(a,b,&c);
        printf("\n(%f+j*%f)-( %f+j*%f)=%f+j*%f\n",
            a.re,a.im,b.re,b.im,c.re,c.im);
    }
}

```



```

    produs(a,b,c);
    printf("\n(%f+j*%f)*(%f+j*%f)=%f+j*%f\n",
           a.re,a.im,b.re,b.im,c.re,c.im);
    impartire(&a,&b,&c);
    printf("\n(%f+j*%f)+(%f+j*%f)=%f+j*%f\n",
           a.re,a.im,b.re,b.im,c.re,c.im);
    printf("\nCONTINUAȚI?DA=D/d,Nu=alt caracter ");
    scanf("%c%c",&ch);
}
}

```

Programul următor prezintă operații asupra datelor de tipul "union":

/ Programul L9Ex2.cpp */*

```

#include <stdio.h>
#include <conio.h>
#include <string.h>
/* Exemplu de folosire a tipului "union" */
void main()
{typedef union{
    char ch[10];
    int x;
    long y;
    float f;
} alfa;

alfa a;
strcpy(a.ch,"ABCDEFGH");
printf("\nDimensiunea zonei de memorie rezervata =%d octeti\n",
       sizeof a);
printf("\nCONTINUTUL ZONEI:\n");
printf("\n-sir de caractere: %s",a.ch);
printf("\n-intreg de tipul int: %d(%x in hexa)",a.x,a.x);
printf("\n-intreg de tipul long: %ld(%lx in hexa)",a.y,a.y);
printf("\n-real de tipul float: %g",a.f);
getch();
}

```

Programul urmator prezinta operatii asupra datelor de tipul "enum".

```
/* Programul L9Ex3.cpp */

#include <stdio.h>
#include <conio.h>
/* Exemplu de folosire a tipului "enum" */
void main()
{
    typedef enum{zero,unu,doi,trei,patru,cinci} NR;
    NR x,y;
    int z,w;
    x=doi;    /* x=2 */
    y=trei;   /* x=3 */
    z=x+y;
    w=x*y;
    printf("\nz=%d w=%d\n",z,w);
    getch();
    x=2;y=3; /* o astfel de atribuire indica "warning" */
    z=x+y;w=x*y;
    printf("\nz=%d w=%d\n",z,w);
    getch();
}
```

3. Mersul lucrării

3.1. Folosind tipul structură pentru o dată curentă an, lună, zi, să se scrie un program pentru a afișa a câtea zi din an este ziua respectivă și câte zile au mai rămas până la sfârșitul anului.

3.2. Folosind tipul structură pentru data de naștere a D-voastră și știind că în anul curent vă aniversați ziua de naștere în ziua de x [luni, marți, ..., duminică], scrieți un program pentru a afișa ziua (din săptămână) în care v-ați născut.

3.3. Să se scrie un program modularizat care citește datele legate de studenții unei grupe: nume, data nașterii, adresa și îi afisează în ordine crescătoare lexicografică.

3.4. Să se scrie un program pentru calculul valorii unui polinom de gradul n cu coeficienți complecși pentru o valoare complexă. Calculul se va face cu ajutorul unei funcții.

3.5. Să se introducă tipul `RAȚIONAL` ca o structură formată din numărător și numitor. Să se scrie funcții de simplificare, adunare, scădere, înmulțire, împărțire, ridicare la putere.

3.6. Folosind tipul uniune, care conține câmpurile necesare pentru a putea reprezenta un cerc, un dreptunghi, un pătrat, un triunghi, să se scrie o funcție pentru a calcula aria figurii respective.

3.7. Folosind tipul enumerare, să se introducă tipul boolean. Să se scrie o funcție de ordonare crescătoare a unui șir folosind metoda bulelor și un semafor de tipul boolean.

3.8. Se citește un șir de caractere format din litere și cifre. Să se indice frecvența de apariție a caracterelor întâlnite în șir folosind o listă ordonată alfabetic (nodul conține caracterul, frecvența și adresa următorului nod).

PRELUCRAREA FIȘIERELOR DE CĂTRE NIVELUL INFERIOR AL S.G.F.

1. Conținutul lucrării

În lucrare sunt prezentate funcțiile de prelucrare a fișierelor la nivelul inferior, adică acelea care fac apel la sistemul de operare. Exemplul prezentat în lucrare ilustrează principalele operații asupra unui fișier: crearea, adăugarea, modificarea și citirea unui fișier.

2. Considerații teoretice

Fișierul este o colecție ordonată de înregistrări, memorate pe un suport extern.

Principalele operații asupra unui fișier sunt:

- deschiderea unui fișier existent;
- crearea unui fișier;
- citirea/scrierea într-un fișier;
- poziționarea într-un fișier;
- ștergerea unui fișier.

Prelucrarea unui fișier se poate face în două moduri:

- a) utilizând funcțiile sistemului de operare (nivelul inferior);
- b) utilizând funcțiile specializate de gestiune a fișierelor (nivelul superior).

Modul de exploatare a unui fișier poate fi:

- secvențial;
- direct.

În lucrare sunt prezentate funcțiile de nivel inferior de prelucrare a fișierelor.

2.1. Crearea unui fișier nou

Pentru a avea un fișier nou, se utilizează funcția `creat`, care are următorul prototip:

```
int creat (const char *calea_nume, int mod);
```

unde:

- `calea_nume` - este un pointer spre un șir de caractere care definește calea de nume (`path_name`) a fișierului care se creează;
- `mod` - este un întreg care poate fi definit prin combinarea cu "**sau pe biți**" a următoarelor drepturi de acces:
 - `S_IREAD` - dreptul de citire;
 - `S_IWRITE` - dreptul de scriere.

Funcția returnează descriptorul de fișier în caz de succes sau -1 în caz de eroare.

Funcția `creat` necesită includerile de fișiere:

```
#include <io.h>
#include <sys/stat.h>
```

Observație importantă: deschiderea în creare a unui fișier existent conduce la ștergerea sa!

Fișierul creat este exploatat în mod text (`O_TEXT`) sau binar (`O_BINARY`), după valoarea variabilei globale `_fmode` (implicit are valoarea `O_TEXT`).

Exemplu:

```
_fmode=O_BINARY;
df=creat ("a:\\FIS.BIN",S_IWRITE);
```

2.2. Deschiderea unui fișier existent

Deschiderea unui fișier existent se face cu ajutorul funcției open, care are următorul prototip:

int open (const char *calea_nume, int acces);

unde:

calea_nume - este un pointer spre un șir de caractere care definește calea de nume a fișierului existent;

acces - este un întreg care este generat prin combinarea pe biți cu ajutorul operatorului "**sau pe biți**" între următoarele constante:

O_RDONLY - deschiderea pentru READ;

O_WRONLY - deschiderea pentru WRITE;

O_RDWR - deschiderea pentru READ și WRITE;

O_APPEND - pointerul în fișier va fi fixat pe sfârșitul de fișier înaintea fiecărei scrieri;

O_TRUNC - dacă fișierul există, el este trunchiat la zero. Drepturile de acces la fișier rămân neschimbate;

O_BINARY - modul de lucru binar;

O_TEXT - modul de lucru caracter (implicit).

Observație: cu ajutorul funcției open, fișierul poate fi creat, caz în care open are prototipul următor:

int open (const char *cale_nume, int acces, unsigned mod);

În acest caz, accesul este o combinație între opțiunile prezentate anterior și următoarele:

O_CREAT - pentru crearea fișierului;

O_EXCL - pentru a returna eroare în caz că fișierul există.

Opțiunile pentru parametrul **mod** sunt identice cu cele de la creat.

Fișierele care trebuie incluse sunt:

#include <fcntl.h>

#include <io.h>

Funcția open returnează descriptorul de fișier sau -1 în caz de eroare.

Exemplu:

```
df=open("C:\\limbaj_c\\FIS.DAT", O_RDWR);
```

Observație: numărul fișierelor deschise la un moment dat este limitat de obicei la 20.

2.3. Citirea dintr-un fișier

Citirea dintr-un fișier existent deschis cu open se face cu ajutorul funcției read, care are următorul prototip:

```
int read (int df, void *buf, unsigned lungime);
```

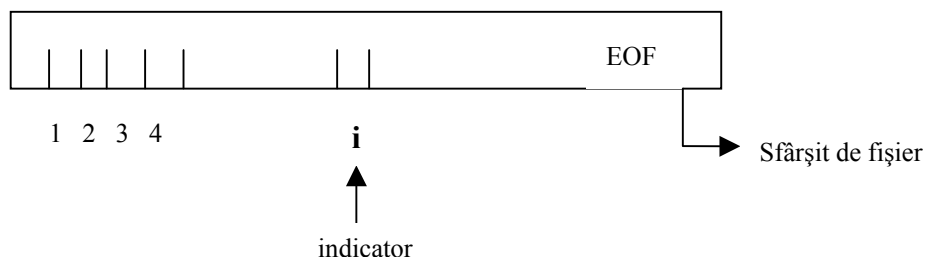
unde:

df - este descriptorul de fișier returnat de open la deschiderea fișierului respectiv;

buf - este pointerul spre zona de memorie în care se păstrează înregistrarea citită din fișier;

lungime - este lungimea în octeți a înregistrării citite.

Fișierul este interpretat ca o succesiune de octeți, începând cu zero. După fiecare citire, indicatorul din fișier indică octetul cu care începe citirea următoare.



La deschidere cu O_RDONLY sau O_RDWR, indicatorul este poziționat pe octetul 0.

Funcția read returnează în caz de succes numărul de octeți efectiv citiți, iar în caz de eroare returnează -1, felul erorii fiind dat de variabila

externă **errno**. În caz de întâlnire a sfârșitului de fișier (CTRL/Z), funcția read returnează valoarea zero.

Fișierul standard de intrare are descriptorul de fișier zero.

Funcția read necesită includerea fișierului io.h.

Exemplu. Citirea a 20 de octeți din fișierul FIS.DAT aflat în directorul curent:

```
df=open ("FIS.DAT", O_RDONLY);  
n=read (df, adr, 20);
```

2.4. Scrierea într-un fișier

Scrierea într-un fișier deschis cu open sau cu creat se face cu ajutorul funcției write, care are ca prototip:

int write (int df, void *buf, unsigned lungime);

unde:

df - este descriptorul de fișier returnat de funcția open sau creat;
buf - este pointerul spre zona de memorie din care se preia înregistrarea care se scrie în fișier;
lungime - este numărul de octeți de scris.

Funcția returnează numărul de octeți efectiv scriși sau -1 în caz de eroare.

Funcția write implică includerea fișierului io.h.

Funcția write poate fi folosită pentru scrierea în fișierul standard, care are df=1;

Exemplu: Scrierea în fișierul FIS.DAT din directorul curent a 20 de octeți de la adresa adr:

```
df=creat ("FIS.DAT", S_IWRITE);  
n=write (df,adr,20);
```


2.5. Poziționarea într-un fișier

În cazul în care se dorește citirea sau scrierea de la o anumită poziție dintr-un fișier pe suport magnetic, se utilizează funcția `lseek`, care are prototipul:

`long lseek (int df, long increment, int origine);`

unde:

- `df` - este descriptorul de fișier deschis;
- `increment` - numărul de octeți peste care se va poziționa indicatorul în fișier, ținând cont de parametrul `origine`;
- `origine` - are una din valorile:
 - 0 - incrementul se consideră față de începutul fișierului;
 - 1 - incrementul se consideră față de poziția curentă a indicatorului de fișier;
 - 2 - incrementul se consideră față de sfârșitul fișierului.

Funcția returnează deplasamentul rezultat față de începutul fișierului sau -1 în caz de eroare. Utilizarea funcției necesită includerea fișierului `io.h`.

Exemple:

a) poziționarea indicatorului la începutul fișierului:

`lseek(df,0l,0);`

b) poziționarea indicatorului la sfârșitul fișierului, caz utilizat pentru adăugarea de noi înregistrări în fișier :

`lseek (df, 0l, 2)`

2.6. Închiderea unui fișier

După terminarea prelucrării unui fișier, acesta se închide apelând funcția `close`, care are prototipul:

`int close(int df);`

`df` fiind descriptorul fișierului.

Funcția returnează zero în caz de succes sau -1 în caz de eroare.

Folosirea funcției necesită includerea fișierului `io.h`.

2.7. Exemplu

În programul următor sunt ilustrate următoarele operații asupra unui fișier:

- crearea fișierului;
- adăugarea de noi înregistrări;
- modificarea unor înregistrări. În acest caz se poziționează indicatorul în fișier pe începutul înregistrării și se scrie noua înregistrare;
- citirea și afișarea conținutului fișierului;
- sortarea fișierului, având drept cheie media, un câmp din înregistrare. Întrucât numărul înregistrărilor unui fișier este mare, s-au citit inițial înregistrările din care s-au extras cheile de sortare, reținându-se numărul de ordine al înregistrărilor și se ordonează în memoria internă. Apoi se obține fișierul sortat.

/* Programul L10Ex1.cpp */

```
#include <stdio.h>
#include <conio.h>
#include <io.h>
#include <fcntl.h>
#include <sys\stat.h>
#include <process.h>
#include <stdlib.h>
/* Exemplu de utilizare a fișierelor */
typedef struct{
    char nume[32];
    float media;
    /*alte informații */
} STUDENT;
typedef union {
    STUDENT stud;
    char st[sizeof(STUDENT)];
} BUF;
typedef struct {
    int nr;
    float med;
} ELEMENT;
```

```

void sortare(char nume_fis[],char nume_fis_sortat[])
{
    ELEMENT el,t[100];
    int k,j,n,i,df1,df2;
    BUF stu;
    j=0;
    df1=open(nume_fis,O_RDONLY);
    while (read(df1,stu.st,sizeof(STUDENT))>0)
    {
        t[j].nr=j;
        t[j].med=stu.stud.media;
        j=j+1;
    };
    /* Sortarea tabloului t după medie */
    n=j-1; /* Elementele tabloului t sunt 0,1,2,...,n */
    j=0;
    do
    {
        k=1;j=j+1; /*k=TRUE*/
        for(i=0;i<=n-j;i++)
            if (t[i].med > t[i+1].med)
            {
                el=t[i];t[i]=t[i+1];t[i+1]=el;
                k=0; /*k=FALSE */
            };
    }
    while(k==0);
    close(df1);
    /*Crearea fişierului sortat */
    df2=creat(nume_fis_sortat,S_IWRITE|S_IREAD);
    df1=open(nume_fis,O_RDONLY);
    for(i=0;i<=n;i++)
    {
        lseek(df1,(long)(t[i].nr * sizeof(STUDENT)),0);
        read(df1,stu.st,sizeof(STUDENT));
        write(df2,stu.st,sizeof(STUDENT));
    };
    close(df1);
}

```

```

        close(df2);
    }

void afisare( char nume_fis[])
{
    BUF stu;
    int j,df1;
    j=0;
    df1=open(nume_fis,O_RDONLY);
    while (read(df1,stu.st,sizeof(STUDENT))>0)
    {
        printf("\n%d %-32s %7.2f", j, stu.stud.numa,
            stu.stud.media);
        j=j+1;
    };
    close(df1);
}

void main()
{
    int i,n,m,df1;long l;
    char ch;
    BUF stu;
    char nume_fis[50]="c:\\ignat\\Limabaj_C\\grupa.dat";
    char nume_fis_sortat[50]="c:\\ignat\\Limabaj_C\\grupasort.dat";
    printf("\nNr.studentilor de introdus n=");
    scanf("%d",&n);
    /*crearea fişierului */
    if ((df1=creat(nume_fis,S_IWRITE|S_IREAD))== -1)
    {
        printf("Nu se poate crea fişierul\n");
        exit(1);
    }
    /* Introducerea datelor despre studenti */
    for(i=1;i<=n;i++)
    {
        printf("\nNumele studentului: ");
        scanf("%s",c);
    }
}

```

```

        gets(stu.stud.num);
        printf("\nMedia=");
        scanf("%f",&stu.stud.media);
        write(df1,stu.st,sizeof(STUDENT));
    };
close(df1); /* Închiderea fişierului */

/* Adăugarea de noi articole in fişier */
printf("\nNr.studentilor de adaugat m=");
scanf("%d",&m);
df1=open(nume_fis,O_RDWR);
lseek(df1,0l,2);
for(i=1;i<=m;i++)
{
    printf("\nNumele studentului adăugat: ");
    scanf("%s",&nume);
    gets(stu.stud.num);
    printf("\nMedia=");
    scanf("%f",&stu.stud.media);
    write(df1,stu.st,sizeof(STUDENT));
};
close(df1); /* Închiderea fişierului */
printf("\n FIŞIERUL DUPA CREARE\n");
afisare(nume_fis);
/* Modificarea datelor din fişier */
printf("\n Modificati? DA=D/d NU= alt caracter ");
scanf("%s",&ch);
df1=open(nume_fis,O_RDWR);
while((ch=='D')||(ch=='d'))
{
    printf("Nr.de ordine al studentului =");
    scanf("%d",&i);
    l=lseek(df1,(long)(sizeof(STUDENT) *i),0);
    printf("depl=%ld pentru i=%d\n",l,i);
    read(df1,stu.st,sizeof(STUDENT));
    printf("\nNumele vechi este:%s\n",stu.stud.num);
    printf("\nNumele modificat:");
    gets(stu.stud.num);
    printf("\n Media veche este: %f",stu.stud.media);
}

```

```

        printf("\nMedia modificată=");
        scanf("%f",&stu.stud.media);
        l=lseek(df1,(long)(sizeof(STUDENT) *i),0);
        printf("depl=%ld pentru i=%d\n",l,i);
        write(df1,stu.st,sizeof(STUDENT));
        printf("\n Mai modificati? DA=D/d NU=alt caracter ");
        scanf("%c",&ch);
    }
    close(df1);
    printf("\nCONȚINUTUL FISIERULUI NESORTAT\n");
    afisare(ume_fis);
    getch();
    printf("\nCONȚINUTUL FISIERULUI SORTAT\n");
    sortare(ume_fis,ume_fis_sortat);
    afisare(ume_fis_sortat);
    getch();
}

```

3. Mersul lucrării

3.1. Se va executa și analiza programul L10Ex1.cpp

3.2. Se citește de la tastatură un text care se scrie într-un fișier "text.dat". Să se afișeze apoi conținutul fișierului, fiecare linie fiind precedată de numărul de ordine al ei.

3.3. De la tastatură se citesc partea reală și partea imaginară pentru n numere complexe. Să se creeze un fișier care conține numerele complexe citite, fiecare număr având partea reală, partea imaginară, modulul și argumentul său.

3.4. Două firme își păstrează informațiile referitoare la stocul de mărfuri (cod produs, denumire, cantitate, preț unitar) în fișierele "marfa1.dat" și respectiv "marfa2.dat", ordonate crescător după cod. Prin fuzionarea celor două firme, rezultă un stoc comun care trebuie memorat în fișierul "marfa.dat", ordonat după cod.

a) Să se creeze fișierele inițiale, pe baza datelor introduse de la tastatură și apoi să se creeze fișierul de stoc comun "marfa.dat" Pentru mărfuri cu cod comun, se consideră că denumirea și prețul unitar corespund.

b) Fișierul "marfa.dat" se va parcurge secvențial, tipărind pentru fiecare componentă denumirea și cantitatea.

c) Pentru o componentă dorită dată prin numărul de ordine, se va modifica direct prețul său unitar.

3.5. Să se scrie programul pentru concatenarea a două sau mai multe fișiere ce conțin numere reale. Se va tipări informația din fișierul rezultat.

PRELUCRAREA FIȘIERELOR DE CĂTRE NIVELUL SUPERIOR AL S.G.F.

1. Conținutul lucrării

În lucrare sunt prezentate funcțiile de prelucrare a fișierelor de nivel superior, utilizând structuri speciale de tip FILE. Principalele operații care se pot efectua asupra fișierelor la acest nivel sunt: crearea, deschiderea, citirea/scrierea unui caracter sau a unui șir de caractere, citirea/scrierea binară a unui număr de articole, poziționarea într-un fișier, închiderea unui fișier, vidarea zonei tampon a unui fișier.

2. Considerații teoretice

La acest nivel, fiecărui fișier *i* se atașează un pointer la o structură de tip FILE:

FILE *p;

Tipul FILE și toate prototipurile funcțiilor de prelucrare se găsesc în fișierul **stdio.h**

2.1. Deschiderea unui fișier

Deschiderea unui fișier existent, precum și crearea unui fișier nou se face cu ajutorul funcției **fopen**, care are următorul prototip:

FILE *fopen(const char *cale_nume, const char *mod);

unde:

- *cale_nume* – este un pointer spre un șir de caractere care definește calea de nume a fișierului;

- **mod** – este un pointer spre un șir de caractere care definește modul de prelucrare a fișierului deschis, după cum urmează:
 - “r” - deschidere în citire (read);
 - “w” - deschidere în scriere (write);
 - “a” - deschidere pentru adăugare (append);
 - “r+” - deschidere în citire/scriere (modificare);
 - “rb” - citire binară;
 - “wb” - scriere binară;
 - “r+b” - citire/scriere binară;
 - „w+b” - citire/scriere binară;
 - “ab” – adăugare de înregistrări în modul binar.

Conținutul unui fișier existent deschis în scriere „w” , va fi șters, el considerându-se deschis în creare.

Dacă fișierul este deschis în modul „a”, se vor putea adăuga noi înregistrări după ultima înregistrare existentă în fișier.

Un fișier inexistent deschis în modul „w” sau „a” va fi creat.

Funcția **fopen** returnează un pointer spre tipul FILE în caz de succes sau pointerul nul în caz de eroare.

Fișierele standard de I/E sunt deschise automat la lansarea programului; pentru ele, pointerii spre tipul FILE sunt:

stdin – intrare standard;
stdout – ieșire standard;
stderr – ieșire standard erori;
stdprn – ieșire paralelă (imprimantă);
stdaux – comunicație serială.

2.2.Prelucrarea pe caractere a unui fișier

Un fișier poate fi prelucrat, în citire sau scriere, caracter cu caracter, folosind funcțiile **getc** și **putc**, ale căror prototipuri sunt:

```
int getc (FILE *pf);  
int putc (int ch, FILE *pf);
```

în care:

- pf este pointerul spre tipul FILE returnat de funcția **fopen**;
- ch este codul ASCII al caracterului care se scrie.

Funcția **getc** returnează codul ASCII al caracterului scris. În caz de eroare ambele returnează -1.

De exemplu, secvența de copiere a intrării standard la ieșirea standard este:

```
while ((c=getc(stdin))!=EOF)
    putc(c, stdout);
```

2.3. Citirea/scrierea unui șir de caractere

Citirea dintr-un fișier a unui șir de caractere se face cu ajutorul funcției **fgets**, care are prototipul:

```
char *fgets(char *s, int n, FILE *pf);
```

în care:

s – este pointerul spre zona din memorie unde are loc păstrarea șirului de caractere;

n – este numărul de octeți a zonei în care se citesc caracterele din fișier. Citirea se oprește la întâlnirea caracterului '\n' sau citirea a cel mult n-1 caractere. Ultimul caracter în ambele cazuri va fi '\0'.

pf – este pointerul spre tipul FILE.

Funcția returnează valoarea pointerului s. La întâlnirea sfârșitului de fișier funcția returnează valoarea zero.

Scrierea unui șir de caractere (inclusiv caracterul „\0”) se face cu funcția **fputs**, care are prototipul:

```
int fputs (const char *s, file *pf);
```

unde s este pointerul spre începutul zonei de memorie care conține șirul de caractere care se scrie în fișier.

Funcția **fputs** returnează codul ASCII al ultimului caracter scris în fișier sau -1 în caz de eroare.

2.4. Citirea/scrierea cu format

Citirea/scrierea cu format se poate face cu ajutorul funcțiilor **fscanf/fprintf**, similare cu funcțiile **scanf/sprintf**, prezentate în lucrarea

L1., deosebirea constând în faptul că în cadrul funcțiilor **sscanf/sprintf** se precizează ca prim parametru pointerul zonei unde se păstrează șirul de caractere, iar în cadrul funcțiilor **fscanf/fprintf** se precizează ca prim parametru pointerul spre tipul FILE, așa cum reiese din prototipurile lor:

```
int fscanf(FILE *pf, const char *format,[adresa,...]);  
int fprintf(FILE *pf, const char *format,[adresa,...]);
```

Funcția **fscanf** returnează numărul de câmpuri citite corect; la întâlnirea sfârșitului de fișier funcția returnează valoarea EOF.

Funcția **fprintf** returnează numărul caracterelor scrise în fișier sau -1 în caz de eroare.

2.5.Vidarea zonei tampon a unui fișier

Vidarea zonei tampon a unui fișier se face cu ajutorul funcției **fflush**, având prototipul următor:

```
int fflush(FILE *pf);
```

unde pf este pointerul spre tipul file.

Dacă fișierul e deschis în scriere, conținutul zonei tampon se scrie în fișierul respectiv.

Dacă fișierul e deschis în citire, caracterele necitite se pierd.

Funcția returnează zero în caz de succes și -1 în caz de eroare.

2.6.Poziționarea într-un fișier

Poziționarea într-un fișier pe un anumit octet se poate face cu ajutorul funcției **fseek**, care are prototipul:

```
int fseek (FILE *pf, long increment, int origine);
```

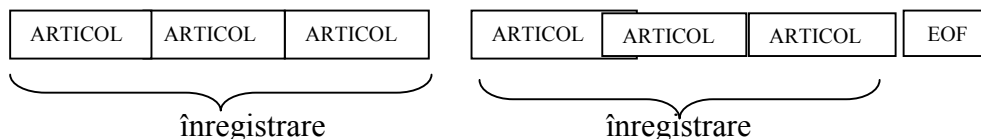
Această funcție este similară cu **lseek**, deosebirea constând în faptul că la **fseek** se precizează pointerul spre tipul FILE, iar la **lseek** se precizează descriptorul de fișier.

Poziția curentă a indicatorului într-un fișier dată prin deplasamentul în octeți față de începutul său este returnată de către funcția **ftell** de prototip:

```
long ftell(FILE *pf);
```

2.7. Prelucrarea fișierelor binare

În acest caz, fișierele sunt considerate ca o succesiune de înregistrări, fiecare înregistrare conținând un număr de articole, ca în figura următoare:



Articolele sunt de lungime fixă. Un articol este o dată de un tip predefinit sau definit de utilizator.

Citirea, respectiv scrierea unei înregistrări se face cu ajutorul funcțiilor **fread** și **fwrite**, care au prototipurile:

```
unsigned fread(void *buf, unsigned dim, unsigned nrart, FILE *pf);  
unsigned fwrite(void *buf, unsigned dim, unsigned nrart, FILE *pf);
```

unde:

buf – este pointerul spre zona tampon care conține articolele citite, respectiv cele care se scriu;

dim – este dimensiunea unui articol în octeți;

nrart – numărul articolelor dintr-o înregistrare;

pf – este pointerul spre tipul FILE.

Funcțiile returnează numărul articolelor citite, respectiv scrise în caz de succes, sau -1 în caz de eroare.

2.8. Închiderea unui fișier

Închiderea unui fișier se realizează cu ajutorul funcției **fclose**, care are prototipul:

```
int fclose(FILE *pf);
```

unde pf este pointerul spre tipul FILE returnat de **fopen**.

Funcția returnează 0 în caz de succes și -1 în caz de eroare.

2.9. Ștergerea unui fișier

Un fișier închis poate fi șters cu ajutorul funcției **unlink**, care are prototipul:

```
int unlink (const char *cale_nume);
```

unde `cale_nume` este un pointer spre un șir de caractere care redă calea de nume a fișierului.

Funcția returnează 0 în caz de succes și -1 în caz de eroare.

2.10. Exemple

Exemplul 1

În programul `L11Ex1.cpp` este creat un fișier caracter cu caracter, citite de la tastatură. Apoi este ilustrat modul de adăugare la sfârșitul fișierului, de data aceasta, a unor șiruri de caractere. La sfârșit fișierul este listat linie cu linie, cu numerotarea lor.

```
/* Programul L11Ex1.cpp */  
  
#include <stdio.h>  
#include <conio.h>  
/* Programul ilustrează prelucrarea fișierului  
    pe caractere și șiruri de caractere */  
void main(void)  
{  
    char ch,s[100],nume_fis[50]="c:\\fis1.txt";  
    int i;  
    FILE *pf;  
    /* crearea fișierului; scrierea caracterelor inclusiv  
        '/n'introduse de la tastatura */  
    pf=fopen(nume_fis,"w");  
    printf("\nIntroduceți textul!\n");  
    while ((ch=getc(stdin))!=EOF)  
    {  
        putc(ch,pf);  
    }  
    fclose(pf);  
    /* Adăugarea de șiruri de caractere*/
```

```

pf=fopen(ume_fis,"r+");
fseek(pf,0l,2);
printf("\nINTRODUCETI sirurile de caractere care se
        adauga terminate cu ENTER \n");
while(fgets(s,100,stdin)!= (char*)0)
{
    fputs(s,pf);
}
fclose(pf);
/*Afisarea continutului */
printf("\nCONTINUTUL FISIERULUI cu NUMEROTAREA
LINIILOR\n");
i=0;
pf=fopen(ume_fis,"r");
while(fgets(s,100,pf)!= (char *)0)
{printf("%d  %s",i,s);
    i++;
}
fclose(pf);
getch();
unlink(ume_fis);
}

```

Exemplul 2

Programul L11Ex2.cpp ilustrează modul de prelucrare binară a unui fișier. Programul conține crearea fișierului și afișarea conținutului său.

```

/* Programul L11Ex2.cpp */

#include <stdio.h>
#include <conio.h>
/* Programul ilustreaza prelucrarea binara a unui fisier */
typedef struct {
    char nume[40];
    long suma;
    /*alte componente */
} ARTICOL;
void afisare(char *ume_fis)

```

```

{
    FILE *pf;
    ARTICOL buf;
    int i;
    pf=fopen(ume_fis,"rb");
    printf("\nNR.CRT.  SUMA  NUMELE-PRENUMELE\n");
    i=0;
    while(fread(&buf,sizeof(ARTICOL),1,pf)>0)
    {
        printf("\n%6d %10ld    %-40s",i,buf.suma,buf.ume);
        i++;
    }
    fclose(pf);
}

void main(void)
{
    FILE *pf;
    ARTICOL buf;
    int i,n;
    char s[40],ume_fis[40]="c:\\fis.dat";
    /*Crearea fisierului */
    printf("\nIntroduceti nr persoanelor n=");
    scanf("%d",&n);
    pf=fopen(ume_fis,"wb");
    for(i=1;i<=n;i++)
    {
        fflush(stdin);
        printf("Numele persoanei: ");
        fgets(buf.ume,40,stdin);
        printf("Suma = ");
        scanf(" %ld",&buf.suma);
        fwrite(&buf,sizeof(ARTICOL),1,pf);
    }
    fclose(pf);
    printf("\nCONTINUTUL FISIERULUI\n");
    afisare(ume_fis);
    getch();
}

```

3. Mersul lucrării

3.1. Se vor analiza și executa exemplele L11Ex1.cpp și L11Ex2.cpp.

3.2 Să se creeze un fișier care să conțină produsele unui magazin.

Un produs este reprezentat printr-o structură ce conține codul produsului, denumirea, unitatea de măsură, cantitatea, prețul unitar.

Plecând de la acest fișier, să se obțină un fișier sortat după cod.

3.3. Având creat fișierul sortat la punctul 3.2. se vor scrie funcții de intrare și de ieșire a produselor magazinului

3.4. Se va scrie un program pentru admiterea la facultate în anul I. Programul va cuprinde crearea fișierului cu candidații înscriși. În final trebuie să se obțină fișierele cu candidații admiși pe secții și cei respinși pe baza mediei obținute ($((\text{bacalaureat} + 2 * \text{test_matem}) / 3.0)$).

3.5. Se consideră un director de fișiere. Fiecare intrare în director conține numele (8 caractere) și extensia (3 caractere) fișierului, numărul de blocuri alocate pentru el, adresa primului bloc alocat, data și ora ultimei actualizări. (zi, luna, an, ora, minut, secunda).

Se cere:

- a) crearea directorului de fișiere;
- b) afișarea directorului în ordine alfabetică a numelor fișierelor;
- c) afișarea directorului în ordine crescătoare a datei și orei ultimei actualizări.

BIBLIOGRAFIE

- 1. V.Cristea, E.Kalisz, I.Atanasiu, V.Iorga. Tehnici de programare. Ed.Teora, București, 1995.**
- 2. V.Dădârlat, I.Ignat, M.Topan, L.Petrescu. Programarea calculatoarelor. Structuri de date și algoritmi. Litografia U.T.C.N., 1995.**
- 3. I.Ignat, V.Dădârlat, M.Topan, L.Petrescu.. Turbo Pascal 7.0. Fundamente și aplicații. Ed.Albastră, Cluj-Napoca, 1996.**
- 4. I.Ignat, V.Dădârlat, I.Rus, A.Kacso. Programarea sistematică în Turbo Pascal 6.0. Ed. Cărții de Știință, Cluj-Napoca, 1992.**
- 5. L.Livovschi, H.Georgescu. Analiza și sinteza algoritmilor. Ed. Enciclopedică, București, 1986.**
- 6. L.Negrescu. Limbajele C și C++ pentru începători, Vol. 1 și 2. Ed. Microinformatica, Cluj_Napoca, 1994 (reeditare 2000).**
- 7. L.D.Serbănați, V.Cristea, F.Moldoveanu, V.Iorga. Programarea sistematică în limbajele Pascal și Fortran. Ed.Tehnică, București, 1984.**
- 8. N.Wirth. Algorithms + Data Structures = Programs.Prentice Hall, 1978.**